

Homework Solution - Classification

I Gusti Ngurah Agung Hari Vijaya Kusuma Batch 57

August 12, 2025

Submission Links

- Repository: github.com/AgungHari

1 Pendahuluan

1.1 Latar Belakang

Perusahaan telekomunikasi di era digital saat ini menghadapi tantangan besar dalam mempertahankan pelanggan. Salah satu permasalahan utama yang sering dihadapi adalah fenomena *customer churn*, yaitu kondisi ketika pelanggan memutuskan untuk berhenti berlangganan suatu layanan. Tingginya tingkat churn dapat berdampak signifikan terhadap pendapatan perusahaan, mengingat biaya untuk memperoleh pelanggan baru umumnya lebih tinggi dibandingkan mempertahankan pelanggan yang sudah ada.

Memahami faktor-faktor yang mempengaruhi churn serta mampu memprediksi pelanggan mana yang berpotensi churn menjadi sangat penting bagi perusahaan. Dengan prediksi yang akurat, perusahaan dapat melakukan intervensi yang tepat, seperti memberikan penawaran khusus atau meningkatkan kualitas layanan, guna mencegah kehilangan pelanggan.

Melalui pemanfaatan data historis pelanggan, seperti data demografi, riwayat penggunaan layanan, dan perilaku pembayaran, teknologi *data science* dan *machine learning* memungkinkan perusahaan untuk membangun model prediksi churn yang efektif. Model ini dapat menjadi alat bantu pengambilan keputusan strategis, sehingga perusahaan dapat fokus pada segmen pelanggan yang paling berisiko churn dan merancang strategi retensi yang lebih efisien.

Tugas ini berfokus pada pembangunan model klasifikasi untuk memprediksi churn pelanggan pada perusahaan telekomunikasi berdasarkan berbagai fitur pelanggan. Diharapkan hasil dari proyek ini dapat memberikan insight dan rekomendasi bisnis yang relevan untuk meningkatkan retensi pelanggan.

1.2 Tujuan

Tujuan dari tugas ini adalah diantara lain untuk:

- Membangun model klasifikasi untuk memprediksi churn pelanggan pada perusahaan telekomunikasi.
- Menganalisis faktor-faktor yang berkontribusi terhadap churn pelanggan.
- Memberikan rekomendasi strategis untuk meningkatkan retensi pelanggan berdasarkan hasil analisis dan model yang dibangun.

1.3 Batasan atau Ruang Lingkup

Batasan masalah dalam tugas ini mencakup:

- Menggunakan dataset yang diberikan oleh rakamin.
- Penggunaan model klasifikasi untuk memprediksi churn.
- Analisis dilakukan pada fitur-fitur yang tersedia dalam dataset, tanpa melakukan pengumpulan data tambahan.

1.4 Manfaat

Manfaat dari tugas ini adalah membuat model klasifikasi yang dapat digunakan untuk memprediksi churn pelanggan.

2 Tinjauan Pustaka

2.1 Customer Churn

Customer churn, atau kehilangan pelanggan, adalah fenomena di mana pelanggan berhenti menggunakan layanan atau produk yang ditawarkan oleh suatu perusahaan. Hal ini menjadi perhatian utama bagi perusahaan, terutama di industri yang sangat kompetitif seperti telekomunikasi, karena churn dapat berdampak langsung pada pendapatan dan profitabilitas.

2.2 EDA (Exploratory Data Analysis)

Exploratory Data Analysis (EDA) adalah proses analisis data yang bertujuan untuk memahami struktur, pola, dan hubungan dalam dataset sebelum menerapkan model statistik atau machine learning. EDA melibatkan visualisasi data, statistik deskriptif, dan identifikasi anomali atau outlier. Proses ini penting untuk mendapatkan wawasan awal tentang data dan membantu dalam pengambilan keputusan selanjutnya.

2.3 Model Klasifikasi

Model klasifikasi adalah teknik dalam machine learning yang digunakan untuk mengelompokkan data ke dalam kategori atau kelas tertentu. Model ini dilatih menggunakan dataset yang berisi fitur-fitur input dan label output yang sesuai. Tujuan dari model klasifikasi adalah untuk memprediksi kelas dari data baru berdasarkan pola yang telah dipelajari dari data pelatihan.

2.4 Logistic Regression

Logistic regression adalah metode statistik yang digunakan untuk analisis regresi ketika variabel dependen bersifat kategorikal. Meskipun namanya mengandung kata "regression", logistic regression sebenarnya digunakan untuk klasifikasi. Metode ini memodelkan probabilitas suatu kejadian dengan menggunakan fungsi logit, yang mengubah output linear menjadi nilai antara 0 dan 1, sehingga cocok untuk prediksi kelas biner.

Untuk memahami logistic regression, kita perlu memahami konsep dasar dari regresi logistik. Regresi logistik digunakan untuk memprediksi probabilitas dari suatu kejadian yang bersifat biner (dua kelas), seperti churn atau tidak churn. Model ini mengasumsikan bahwa log odds dari probabilitas kejadian tersebut adalah linear terhadap fitur-fitur input.

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}} \quad (1)$$

di mana:

- $P(Y = 1|X)$ adalah probabilitas bahwa kelas target Y adalah 1 (misalnya, churn).
- β_0 adalah intercept dari model.
- $\beta_1, \beta_2, \dots, \beta_n$ adalah koefisien regresi untuk fitur $X_1, X_2, \dots, X_n$.
- e adalah basis dari logaritma natural.
- $X_1, X_2, \dots, X_n$ adalah fitur-fitur input yang digunakan untuk memprediksi kelas target.

Dalam tugas ini apabila perhitungan probabilitas $P(Y=1|X)$ lebih besar dari 0.5 maka akan dikategorikan sebagai churn, sebaliknya jika kurang dari 0.5 maka tidak churn.

2.5 Random Forest

Random Forest adalah algoritma ensemble learning yang menggabungkan beberapa pohon keputusan (decision trees) untuk meningkatkan akurasi prediksi. Setiap pohon dalam hutan dibangun menggunakan subset acak dari data pelatihan dan fitur, sehingga mengurangi risiko overfitting yang sering terjadi pada pohon keputusan tunggal. Random Forest dapat digunakan untuk klasifikasi maupun regresi, dan memiliki keunggulan dalam menangani data besar dengan banyak fitur.

$$\hat{Y} = \frac{1}{N} \sum_{i=1}^N T_i(X) \quad (2)$$

di mana:

- $\hat{Y}$ adalah prediksi akhir dari Random Forest.
- N adalah jumlah pohon dalam hutan.
- $T_i(X)$ adalah prediksi dari pohon keputusan ke- i untuk input X .
- X adalah fitur-fitur input yang digunakan untuk memprediksi kelas target.
- $\sum_{i=1}^N T_i(X)$ adalah jumlah prediksi dari semua pohon dalam hutan.

2.6 XGBoost/Gradient Boosting

XGBoost (Extreme Gradient Boosting) adalah algoritma machine learning yang merupakan implementasi dari teknik gradient boosting. XGBoost dirancang untuk efisiensi, fleksibilitas, dan kinerja yang tinggi. Algoritma ini membangun model prediksi secara bertahap dengan menambahkan pohon keputusan baru yang mengoreksi kesalahan dari pohon sebelumnya. XGBoost sangat populer dalam kompetisi data science karena kemampuannya dalam menangani data besar dan kompleks.

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (3)$$

di mana:

- $\hat{y}_i$ adalah prediksi untuk data ke-i.
- K adalah jumlah pohon dalam model XGBoost.
- $f_k(x_i)$ adalah output dari pohon ke-k untuk input x_i .
- x_i adalah fitur-fitur input yang digunakan untuk memprediksi kelas target.
- $\sum_{k=1}^K f_k(x_i)$ adalah jumlah prediksi dari semua pohon dalam model XGBoost.

2.7 Confusion Matrix

Confusion matrix merupakan salah satu pengukuran yang paling mudah dilakukan dalam mencari nilai tingkat kebenaran dan juga akurasi dari model. *Confusion matrix* adalah sebuah tabel berbentuk dua dimensi yang terdiri dari data aktual dan data prediksi yang masing-masing memiliki kelas. Data aktual terletak pada bagian kolom tabel, sedangkan data prediksi terletak pada bagian baris dari tabel. Gambar 1 merupakan representasi visual dari perhitungan confusion matrix.

		Predicted Class	
		1 (Positive)	0 (Negative)
Actual Class	1 (Positive)	TP (True Positive)	FN (False Negative) Type II Error
	0 (Negative)	FP (False Positive) Type I Error	TN (True Negative)

Gambar 1: Contoh Confusion Matrix

Confusion matrix memberikan informasi tentang jumlah prediksi yang benar dan salah untuk setiap kelas. Dari confusion matrix, kita dapat menghitung berbagai metrik evaluasi model seperti akurasi, presisi, recall, dan F1-score. Metrik-metrik ini membantu dalam menilai kinerja model klasifikasi secara keseluruhan.

2.8 Akurasi

Akurasi adalah metrik evaluasi yang mengukur seberapa baik model klasifikasi dalam memprediksi kelas yang benar. Akurasi didefinisikan sebagai rasio antara jumlah prediksi benar (true positives + true negatives) dengan jumlah total prediksi. Rumusnya adalah:

$$\text{Akurasi} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Prediksi}} \quad (4)$$

di mana:

- True Positives (TP): Jumlah kasus positif yang benar-benar terdeteksi oleh model.
- True Negatives (TN): Jumlah kasus negatif yang benar-benar terdeteksi oleh model.
- Total Prediksi: Jumlah total kasus yang diprediksi oleh model, yaitu jumlah dari true positives, true negatives, false positives, dan false negatives.

Akurasi memberikan gambaran umum tentang kinerja model, tetapi dapat menjadi menyesatkan jika dataset tidak seimbang (misalnya, jika satu kelas jauh lebih banyak daripada kelas lainnya). Oleh karena itu, penting untuk mempertimbangkan metrik lain seperti recall dan precision untuk mendapatkan gambaran yang lebih lengkap tentang kinerja model.

2.9 Recall

Recall, juga dikenal sebagai sensitivitas atau true positive rate, adalah metrik evaluasi yang mengukur kemampuan model dalam mengidentifikasi kelas positif. Recall didefinisikan sebagai rasio antara jumlah prediksi benar positif (true positives) dengan jumlah total kasus positif aktual (true positives + false negatives). Rumusnya adalah:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (5)$$

di mana:

- True Positives (TP): Jumlah kasus positif yang benar-benar terdeteksi oleh model.
- False Negatives (FN): Jumlah kasus positif yang tidak terdeteksi oleh model (salah diklasifikasikan sebagai negatif).

Recall sangat penting dalam konteks di mana kita ingin meminimalkan jumlah kasus positif yang terlewatkan, seperti dalam deteksi penyakit atau pencegahan churn pelanggan. Metrik ini memberikan gambaran tentang seberapa baik model dapat menangkap semua kasus positif yang sebenarnya ada.

2.10 Precision

Precision adalah metrik evaluasi yang mengukur akurasi dari prediksi positif yang dibuat oleh model. Precision didefinisikan sebagai rasio antara jumlah prediksi benar positif (true positives) dengan jumlah total prediksi positif (true positives + false positives). Rumusnya adalah:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (6)$$

di mana:

- True Positives (TP): Jumlah kasus positif yang benar-benar terdeteksi oleh model.
- False Positives (FP): Jumlah kasus negatif yang salah diklasifikasikan sebagai positif oleh model.

Precision sangat penting dalam konteks di mana kita ingin meminimalkan jumlah prediksi positif yang salah, seperti dalam deteksi penipuan atau spam. Metrik ini memberikan gambaran tentang seberapa akurat model dalam mengidentifikasi kasus positif yang sebenarnya ada.

2.11 F1-Score

F1-Score adalah metrik evaluasi yang menggabungkan precision dan recall menjadi satu nilai tunggal. F1-Score dihitung sebagai harmonik rata-rata dari precision dan recall, sehingga memberikan keseimbangan antara kedua metrik tersebut. F1-Score sangat berguna ketika kita ingin mempertimbangkan trade-off antara precision dan recall, terutama dalam situasi di mana dataset tidak seimbang.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (7)$$

di mana:

- Precision adalah rasio antara true positives dan total prediksi positif.
- Recall adalah rasio antara true positives dan total kasus positif aktual.
- F1-Score memberikan nilai antara 0 dan 1, di mana nilai 1 menunjukkan keseimbangan sempurna antara precision dan recall, sedangkan nilai 0 menunjukkan kinerja yang buruk.

F1-Score sangat berguna dalam konteks di mana kita ingin meminimalkan baik false positives maupun false negatives, seperti dalam deteksi penyakit atau klasifikasi teks. Metrik ini memberikan gambaran yang lebih komprehensif tentang kinerja model dibandingkan dengan precision atau recall secara terpisah.

2.12 ROC-AUC

ROC (Receiver Operating Characteristic) curve adalah grafik yang menunjukkan kinerja model klasifikasi biner pada berbagai ambang batas (threshold). ROC curve memplot true positive rate (TPR) terhadap false positive rate (FPR) pada berbagai nilai threshold. Area di bawah kurva ROC (AUC - Area Under the Curve) memberikan ukuran kinerja model secara keseluruhan.

AUC adalah nilai antara 0 dan 1, di mana nilai 1 menunjukkan model yang sempurna (mampu memisahkan kelas positif dan negatif dengan sempurna), sedangkan nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak. AUC yang lebih tinggi menunjukkan bahwa model memiliki kemampuan yang lebih baik dalam membedakan antara kelas positif dan negatif.

$$AUC = \int_0^1 TPR(FPR) dFPR \quad (8)$$

di mana:

- TPR (True Positive Rate) adalah rasio antara jumlah prediksi benar positif dengan jumlah total kasus positif aktual.
- FPR (False Positive Rate) adalah rasio antara jumlah prediksi salah positif dengan jumlah total kasus negatif aktual.
- Integral ini menghitung area di bawah kurva ROC, yang memberikan nilai AUC.
- Nilai AUC yang lebih tinggi menunjukkan bahwa model memiliki kemampuan yang lebih baik dalam membedakan antara kelas positif dan negatif.

2.13 Chi-Squared Test

Chi-Squared Test adalah metode statistik yang digunakan untuk menguji independensi antara dua variabel kategorikal. Uji ini membandingkan frekuensi yang diamati dalam kategori tertentu dengan frekuensi yang diharapkan jika kedua variabel tersebut independen. Chi-Squared Test sering digunakan dalam analisis data untuk menentukan apakah ada hubungan signifikan antara dua variabel.

Chi-Squared Test menghitung nilai statistik Chi-Squared (χ^2) berdasarkan perbedaan antara frekuensi yang diamati (O_i) dan frekuensi yang diharapkan (E_i). Rumusnya adalah:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (9)$$

di mana:

- χ^2 adalah nilai statistik Chi-Squared.
- O_i adalah frekuensi yang diamati untuk kategori ke-i.
- E_i adalah frekuensi yang diharapkan untuk kategori ke-i jika kedua variabel independen.
- Penjumlahan dilakukan untuk semua kategori yang ada.
- Nilai χ^2 yang lebih tinggi menunjukkan bahwa ada perbedaan signifikan antara frekuensi yang diamati dan yang diharapkan, yang mengindikasikan bahwa kedua variabel mungkin tidak independen.

2.14 Cramer's V

Cramer's V adalah ukuran asosiasi antara dua variabel kategorikal yang digunakan untuk mengukur kekuatan hubungan antara variabel tersebut. Nilai Cramer's V berkisar antara 0 hingga 1, di mana 0 menunjukkan tidak ada asosiasi dan 1 menunjukkan asosiasi sempurna. Cramer's V dihitung berdasarkan nilai Chi-Squared dan ukuran sampel.

Asosiasi adalah hubungan antara dua variabel yang dapat diukur dengan berbagai cara. Cramer's V adalah salah satu metode yang digunakan untuk mengukur kekuatan asosiasi antara dua variabel kategorikal. Nilai Cramer's V berkisar antara 0 hingga 1, di mana 0 menunjukkan tidak ada asosiasi dan 1 menunjukkan asosiasi sempurna.

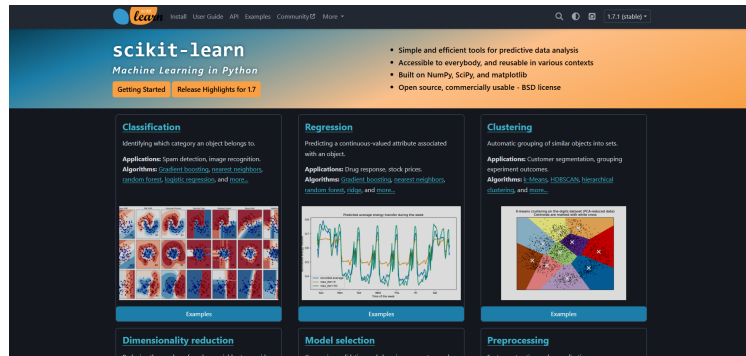
$$V = \sqrt{\frac{\chi^2}{n \cdot \min(k-1, r-1)}} \quad (10)$$

di mana:

- V adalah nilai Cramer's V.
- χ^2 adalah nilai statistik Chi-Squared yang dihitung dari tabel kontingensi.
- n adalah ukuran sampel (jumlah total observasi).
- k adalah jumlah kategori pada variabel pertama.
- r adalah jumlah kategori pada variabel kedua.
- Nilai Cramer's V yang lebih tinggi menunjukkan asosiasi yang lebih kuat antara kedua variabel kategorikal.
- Jika $V = 0$, berarti tidak ada asosiasi antara kedua variabel, sedangkan jika $V = 1$, berarti ada asosiasi sempurna.

2.15 Scikit-learn

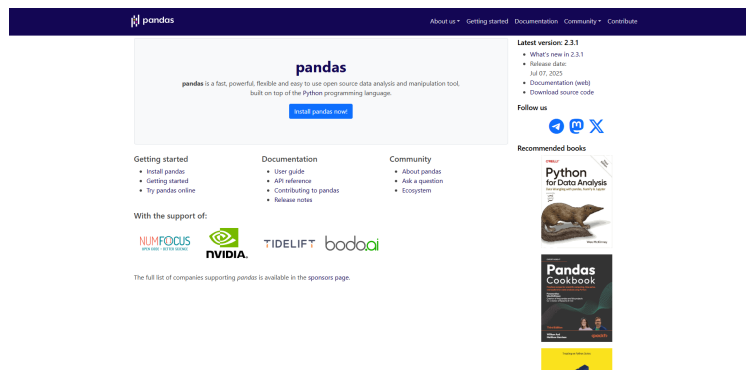
scikit learn adalah pustaka Python yang menyediakan berbagai algoritma machine learning dan alat untuk analisis data. Pustaka ini dirancang untuk kemudahan penggunaan dan integrasi dengan pustaka lain seperti NumPy dan pandas. scikit-learn mencakup berbagai algoritma untuk klasifikasi, regresi, clustering, dan pengurangan dimensi, serta alat untuk evaluasi model dan pemrosesan data.



Gambar 2: Dokumentasi scikit-learn

2.16 Pandas

Pandas adalah pustaka Python yang menyediakan struktur data dan alat analisis data yang efisien. Pustaka ini dirancang untuk memudahkan manipulasi dan analisis data, terutama dalam bentuk tabel (DataFrame). Pandas menyediakan berbagai fungsi untuk membaca, menulis, dan memanipulasi data, serta alat untuk melakukan operasi statistik dan agregasi.



Gambar 3: Dokumentasi Pandas

3 Desain dan Implementasi

Tugas ini dilakukan sesuai dengan desain sistem berikut beserta implementasinya. Desain sistem adalah konsep dari pembuatan dan perancangan infrastruktur dan kemudian diwujudkan dalam bentuk alur yang harus dikerjakan

3.1 Deskripsi sistem

Pada tugas ini kita akan melakukan analisis data dan membangun model klasifikasi untuk memprediksi apakah pelanggan perusahaan telekomunikasi penyedia wifi akan berhenti berlangganan atau tidak. Dataset yang digunakan adalah dataset churn pelanggan yang berisi informasi tentang pelanggan, seperti jenis layanan, jenis kelamin, status pekerjaan, biaya langganan, dan informasi lainnya. Desain sistem ini mencakup langkah-langkah yang akan dijabarkan dalam Gambar 4.



Gambar 4: Desain Sistem

3.2 Feature Extraction

Feature extraction adalah proses mengidentifikasi dan memilih fitur-fitur yang relevan dari dataset yang akan digunakan dalam model klasifikasi. Dalam tugas ini, fitur-fitur yang dipilih mencakup informasi tentang pelanggan, seperti jenis layanan, jenis kelamin, status pekerjaan, biaya langganan, dan informasi lainnya.

3.2.1 Deskripsi Fitur

Dataset yang digunakan dalam tugas ini adalah dataset dalam format .csv yang berisi informasi langganan internet dari perusahaan telekomunikasi. Dataset ini mencakup berbagai informasi seperti customerid, jenis layanan, jenis kelamin, status pekerjaan, biaya langganan, dan informasi lainnya. Setiap baris dalam dataset mewakili satu pelanggan dan setiap kolom mewakili fitur-fitur tersebut. Gambar 5 merupakan contoh dari dataset yang digunakan.

customerid	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn	
7590-WWEG	Female	0	Yes	No	1	No	No	phone service	DSL	No	Yes	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No	
5575-GNVE	Male	0	No	No	34	Yes	No	DSL	Yes	No	No	No	One year	No	Mailed check	36.95	1889.5	No			
3668-GPYB	Male	0	No	No	2	Yes	No	DSL	Yes	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes			
7795-CFCW	Male	0	No	No	45	No	No	phone service	DSL	Yes	No	Yes	No	No	One year	No	Bank transfer (automatic)	42.3	1840.75	No	
9237-HQTL	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No	Month-to-month	Yes	Electronic check	78.7	151.45	Yes			
9305-CDSC	Female	0	No	No	8	Yes	Yes	Fiber optic	No	No	Yes	No	Yes	Month-to-month	Yes	Electronic check	99.65	820.5	Yes		
1453-KIOVL	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No	Yes	No	No	Month-to-month	Yes	Credit card (automatic)	89.1	1949.4	No			
8713-CKDM	Female	0	No	No	10	No	No	phone service	DSL	No	No	No	Month-to-month	No	Mailed check	29.75	301.3	No			
7892-POOK	Female	0	Yes	No	25	Yes	Yes	Fiber optic	No	No	Yes	Yes	Month-to-month	Yes	Electronic check	104.8	3046.05	Yes			
6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	56.15	3487.95	No			
9783-GRSD	Male	0	Yes	Yes	13	Yes	No	DSL	Yes	No	No	No	Month-to-month	Yes	Mailed check	49.95	587.45	No			
7469-LBKC	Male	0	No	No	16	Yes	No	No	Internet service	No	Internet service	No	Internet service	No	Internet service	Two year	No	Credit card (automatic)	18.95	326.8	No
8091-TTVA	Male	0	Yes	No	58	Yes	Yes	Fiber optic	No	No	Yes	No	One year	No	Credit card (automatic)	100.35	5681.1	No			
0280-XJGX	Male	0	No	No	49	Yes	Yes	Fiber optic	No	Yes	No	Yes	Month-to-month	Yes	Bank transfer (automatic)	103.7	5036.3	Yes			

Gambar 5: Contoh Dataset Pelanggan

Adapun untuk memperjelas fitur-fitur yang akan dipilih dalam metode feature selection untuk model klasifikasi kita nantinya, berikut adalah deskripsi singkat dari beberapa fitur yang terdapat dalam dataset:

- Customerid: ID unik dari pelanggan.
- Gender: Jenis kelamin pelanggan (Pria/Wanita).
- Seniorcitizen: Apakah pelanggan merupakan senior citizen (1 untuk ya, 0 untuk tidak).
- Partner: Apakah pelanggan memiliki pasangan (Ya/Tidak).
- Dependents: Apakah pelanggan memiliki tanggungan (Ya/Tidak).
- Tenure: Lama berlangganan layanan (dalam bulan).
- PhoneService: Apakah pelanggan menggunakan layanan telepon (Ya/Tidak).
- MultipleLines: Apakah pelanggan menggunakan layanan telepon multiple lines (Ya/Tidak).
- InternetService: Tipe layanan internet yang digunakan (DSL/Fiber optic/No).
- OnlineSecurity: Apakah pelanggan menggunakan fitur keamanan online (Ya/Tidak).
- OnlineBackup: Apakah pelanggan menggunakan fitur backup online (Ya/Tidak).
- DeviceProtection: Apakah pelanggan menggunakan fitur perlindungan perangkat (Ya/Tidak).
- TechSupport: Apakah pelanggan menggunakan fitur dukungan teknis (Ya/Tidak).
- StreamingTV: Apakah pelanggan menggunakan fitur streaming TV (Ya/Tidak).
- StreamingMovies: Apakah pelanggan menggunakan fitur streaming film (Ya/Tidak).
- Contract: Tipe kontrak layanan (Month-to-month/One year/Two year).
- PaperlessBilling: Apakah pelanggan menggunakan penagihan tanpa kertas (Ya/Tidak).
- PaymentMethod: Metode pembayaran yang digunakan (Electronic check / Mailed check / Bank transfer (automatic) /Credit card (automatic)).
- MonthlyCharges: Biaya bulanan yang dibayarkan oleh pelanggan.
- TotalCharges: Total biaya yang dibayarkan oleh pelanggan selama berlangganan.
- Churn: Target variabel yang menunjukkan apakah pelanggan berhenti berlangganan (Ya/Tidak), dan merupakan Label target yang akan diprediksi oleh model klasifikasi.

3.3 EDA (Exploratory Data Analysis)

Exploratory Data Analysis (EDA) adalah proses analisis awal terhadap dataset untuk memahami struktur, pola, dan hubungan antar fitur. Dalam tugas ini, EDA dilakukan untuk mengidentifikasi karakteristik data, mendeteksi adanya missing values, outliers, dan distribusi dari setiap fitur.

Pertama tama yang kita akan lakukan adalah mengimport dataset dan menyiapkan library yang akan digunakan dalam EDA. Pandas, NumPy, Matplotlib, dan Seaborn adalah beberapa library yang umum digunakan untuk analisis data dan visualisasi di Python. Pandas digunakan untuk manipulasi data, NumPy untuk operasi numerik, Matplotlib dan Seaborn untuk visualisasi data.

Listing 1: Import Library dan Dataset

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Import dataset
7 df = pd.read_csv('dataset.csv')
```

Setelah mengimport dataset, langkah selanjutnya adalah melakukan analisis awal terhadap data. Kita akan melihat beberapa informasi dasar tentang dataset, seperti jumlah baris dan kolom, tipe data dari setiap kolom, serta beberapa baris pertama dari dataset. Dengan menggunakan `.info` kita bisa melihat tipe data dari setiap kolom.

Listing 2: Informasi Awal Dataset

```
1 # Informasi awal dataset
2 df.info()
3
4 # Output
5 <class 'pandas.core.frame.DataFrame'>
6 RangeIndex: 7043 entries, 0 to 7042
7 Data columns (total 21 columns):
8 #    Column                Non-Null Count  Dtype
9 ---  -
10 0    customerID             7043 non-null   object
11 1    gender                 7043 non-null   object
12 2    SeniorCitizen          7043 non-null   int64
13 3    Partner                7043 non-null   object
14 4    Dependents             7043 non-null   object
15 5    tenure                 7043 non-null   int64
16 6    PhoneService           7043 non-null   object
17 7    MultipleLines          7043 non-null   object
18 8    InternetService        7043 non-null   object
19 9    OnlineSecurity         7043 non-null   object
20 10   OnlineBackup            7043 non-null   object
21 11   DeviceProtection       7043 non-null   object
```

```

22 12 TechSupport      7043 non-null object
23 13 StreamingTV     7043 non-null object
24 14 StreamingMovies  7043 non-null object
25 15 Contract         7043 non-null object
26 16 PaperlessBilling 7043 non-null object
27 17 PaymentMethod    7043 non-null object
28 18 MonthlyCharges   7043 non-null float64
29 19 TotalCharges     7043 non-null object
30 20 Churn            7043 non-null object
31 dtypes: float64(1), int64(2), object(18)
32 memory usage: 1.1+ MB

```

Setelah saya analisa ternyata terdapat beberapa kolom yang memiliki tipe data yang tidak sesuai, seperti kolom TotalCharges yang seharusnya bertipe numerik namun masih bertipe object. Hal ini bisa terjadi karena adanya missing values atau karakter non-numerik dalam kolom tersebut. Oleh karena itu, kita perlu melakukan pembersihan data untuk memastikan bahwa semua kolom memiliki tipe data yang sesuai.

Listing 3: Perbaikan Tipe Data Kolom TotalCharges

```

1 # Mengubah tipe data kolom TotalCharges menjadi numerik
2 df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
3
4 # Mengecek missing values setelah perubahan tipe data
5 missing_values = df.isnull().sum()
6 print("Missing values after type conversion:\n", missing_values)
7
8 # Output
9 customerID      0
10 gender          0
11 .....
12 .....
13 PaperlessBilling 0
14 PaymentMethod   0
15 MonthlyCharges  0
16 TotalCharges    11
17 Churn           0
18 dtype: int64

```

Saya juga melakukan pengecekan terhadap missing values pada dataset. Dengan menggunakan `.isnull().sum()` kita bisa melihat jumlah missing values pada setiap kolom. Dari hasil pengecekan, terdapat 11 missing values pada kolom TotalCharges. Kita perlu menangani missing values ini sebelum melanjutkan analisis lebih lanjut.

Untuk menangani missing values pada kolom TotalCharges, kita bisa menghapus baris yang memiliki missing values tersebut atau mengisi missing values dengan nilai rata-rata atau median dari kolom tersebut. Dalam tugas ini, saya memilih untuk menghapus baris yang memiliki missing values.

Listing 4: Menghapus Missing Values

```
1 # output
2 .....
3 .....
4 MonthlyCharges      0
5 TotalCharges        0
6 Churn               0
7 dtype: int64
```

Kita telah selesai menghapus missing values pada kolom TotalCharges. Setelah itu, kita akan melakukan analisis lebih lanjut terhadap fitur-fitur yang ada dalam dataset. Kita akan melihat distribusi dari setiap fitur, hubungan antar fitur, serta outliers yang mungkin ada dalam dataset.

Agar memudahkan analisa maka dataset akan dibagi menjadi dua bagian, yaitu dataset untuk fitur numerik dan dataset untuk fitur kategorikal. Hal ini dilakukan agar analisis dapat dilakukan dengan lebih efektif.

Adapun pembagiannya sebagai berikut :

Listing 5: Memisahkan Fitur Numerik dan Kategorikal

```
1 Numerical Columns: ['tenure', 'MonthlyCharges', 'TotalCharges']
2 Categorical Columns: ['customerID', 'gender', 'SeniorCitizen',
3 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
4 'InternetService', 'OnlineSecurity', 'OnlineBackup',
5 'DeviceProtection', 'TechSupport', 'StreamingTV',
6 'StreamingMovies', 'Contract', 'PaperlessBilling',
7 'PaymentMethod', 'Churn']
```

3.3.1 Analisis Fitur Kategorikal

Analisis fitur kategorikal dilakukan untuk memahami distribusi dari setiap fitur kategorikal dalam dataset. Kita akan menggunakan visualisasi seperti bar plot untuk melihat jumlah pelanggan berdasarkan setiap kategori. Namun agar barplot tidak terlalu banyak kita akan uji dengan chi square terlebih dahulu untuk melihat fitur mana saja yang signifikan terhadap target variabel Churn. Dengan begitu kita bisa plot barplot hanya untuk fitur yang signifikan saja.

Setelah melakukan pengecekan menggunakan chi-square saya menemukan bahwa terdapat fitur-fitur yang signifikan terhadap target variabel Churn. Adapun Tabel 1 berikut adalah hasil dari uji chi-square yang dilakukan.

Table 1: Hasil Uji Chi-Square

Nama Fitur	Chi-Squared	P-Value	Significant Association	Cramer's V
Contract	1179.55	7.33×10^{-257}	Ya	0.4096
TechSupport	824.93	7.41×10^{-180}	Ya	0.3425
OnlineSecurity	846.68	1.40×10^{-184}	Ya	0.3470
InternetService	728.70	5.83×10^{-159}	Ya	0.3219
PaymentMethod	645.43	1.43×10^{-139}	Ya	0.3030
OnlineBackup	599.18	7.78×10^{-131}	Ya	0.2919
DeviceProtection	555.88	1.96×10^{-121}	Ya	0.2812
StreamingMovies	374.27	5.35×10^{-82}	Ya	0.2307
StreamingTV	372.46	1.32×10^{-81}	Ya	0.2301
Dependents	186.32	2.02×10^{-42}	Ya	0.1628
SeniorCitizen	158.44	2.48×10^{-36}	Ya	0.1501
Partner	157.50	3.97×10^{-36}	Ya	0.1497
MultipleLines	11.27	0.0036	Ya	0.0400
PhoneService	0.87	0.35	Tidak	0.0111
Gender	0.48	0.49	Tidak	0.0082

Dapat dilihat bahwa fitur yang memiliki nilai Chi-Squared yang tinggi dan p-value yang sangat kecil menunjukkan adanya hubungan yang signifikan dengan target variabel Churn. Fitur-fitur seperti Contract, TechSupport, OnlineSecurity, dan InternetService memiliki nilai Cramer's V yang cukup tinggi, menunjukkan bahwa fitur-fitur tersebut memiliki pengaruh yang signifikan terhadap keputusan pelanggan untuk berhenti berlangganan.

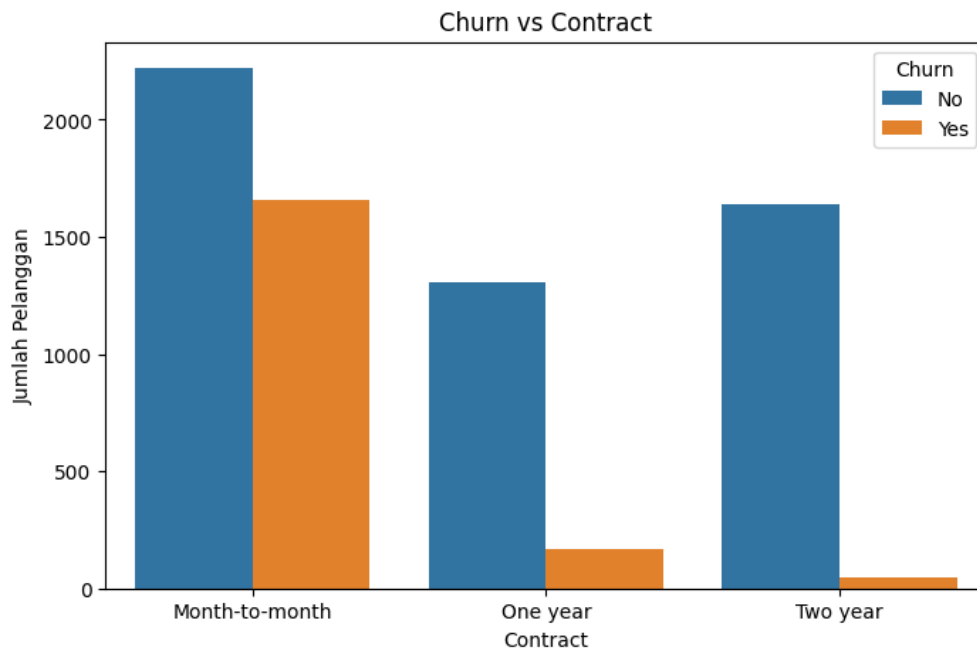
Sedangkan fitur dengan nilai Chi-Squared yang rendah dan p-value yang tinggi menunjukkan kebalikannya yaitu tidak ada hubungan yang signifikan terhadap label target variabel churn. Fitur-fitur seperti PhoneService dan Gender tidak menunjukkan hubungan yang signifikan dengan target variabel Churn, sehingga kita tidak perlu melakukan analisis lebih lanjut terhadap fitur-fitur tersebut.

Setelah mengetahui fitur-fitur yang signifikan, kita akan melakukan visualisasi untuk melihat distribusi dari setiap fitur kategorikal yang signifikan terhadap target variabel Churn. Kita akan menggunakan bar plot untuk melihat jumlah pelanggan berdasarkan setiap kategori.

Adapun pertanyaan yang ingin saya jawab untuk analisis fitur kategorikal ini adalah sebagai berikut:

- Apakah pelanggan yang dengan contract kerja tertentu cenderung churn?
- Apakah pelanggan yang memilih langganan dengan paket bantuan TechSupport cenderung churn?
- Apakah pelanggan yang memilih layanan keamanan online cenderung churn?
- Apakah pelanggan yang memilih layanan internet tertentu cenderung churn?
- Apakah pelanggan yang memilih metode pembayaran tertentu cenderung churn?

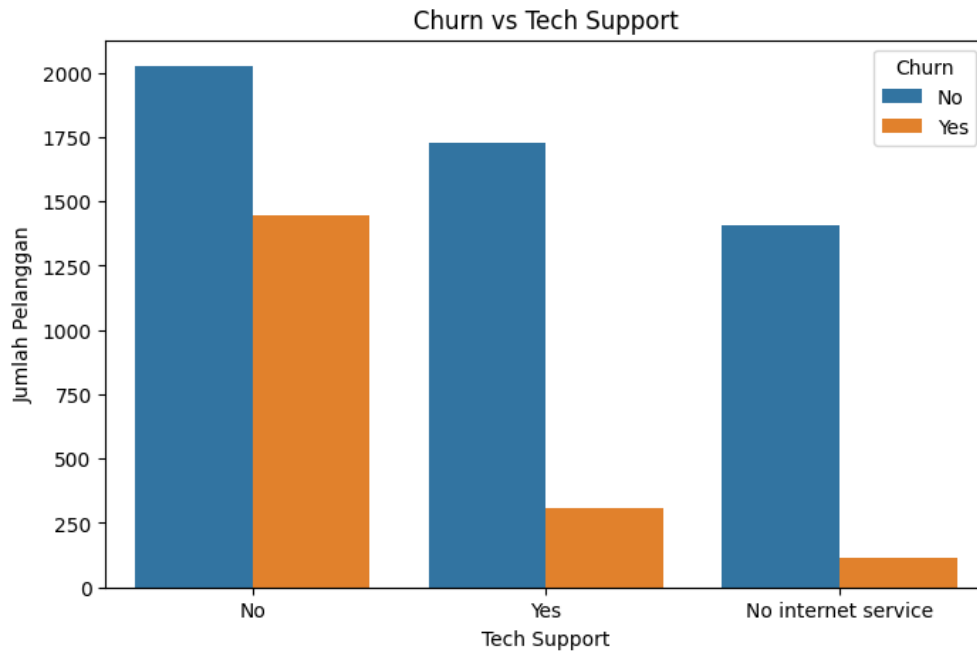
Untuk menjawab pertanyaan pertama yaitu apakah pelanggan yang dengan contract kerja tertentu cenderung churn, kita akan membuat bar plot untuk melihat jumlah pelanggan berdasarkan jenis kontrak. Kita akan membandingkan jumlah pelanggan yang churn dan tidak churn berdasarkan jenis kontrak. Dengan menggunakan `.barplot()` dari Seaborn, kita bisa membuat visualisasi yang jelas dan mudah dipahami. Gambar 6 menunjukkan hasil dari visualisasi tersebut.



Gambar 6: Jumlah Pelanggan Churn Berdasarkan Jenis Kontrak

Dari Gambar 6 dapat dilihat bahwa pelanggan dengan kontrak "Month-to-month" memiliki jumlah churn yang paling tinggi dibandingkan dengan kontrak "One year" dan "Two year". Hal ini menunjukkan bahwa pelanggan yang memilih kontrak bulanan cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang memilih kontrak tahunan atau dua tahunan.

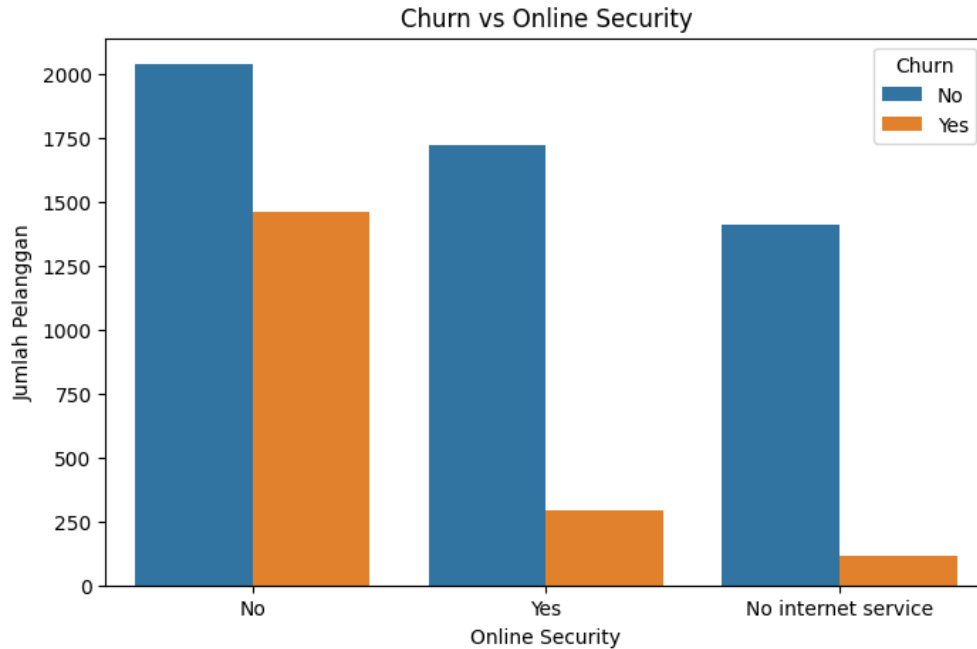
Untuk pertanyaan kedua yaitu apakah pelanggan yang memilih langganan dengan paket bantuan TechSupport cenderung churn, kita akan membuat bar plot untuk melihat jumlah pelanggan berdasarkan pilihan TechSupport. Kita akan membandingkan jumlah pelanggan yang churn dan tidak churn berdasarkan pilihan TechSupport. Gambar 7 menunjukkan hasil dari visualisasi tersebut.



Gambar 7: Jumlah Pelanggan Churn Berdasarkan Pilihan TechSupport

Dari Gambar 7 dapat dilihat bahwa pelanggan yang memilih untuk menggunakan layanan TechSupport memiliki jumlah churn yang lebih rendah dibandingkan dengan pelanggan yang tidak menggunakan layanan TechSupport. Hal ini menunjukkan bahwa layanan TechSupport dapat membantu mengurangi tingkat churn pelanggan.

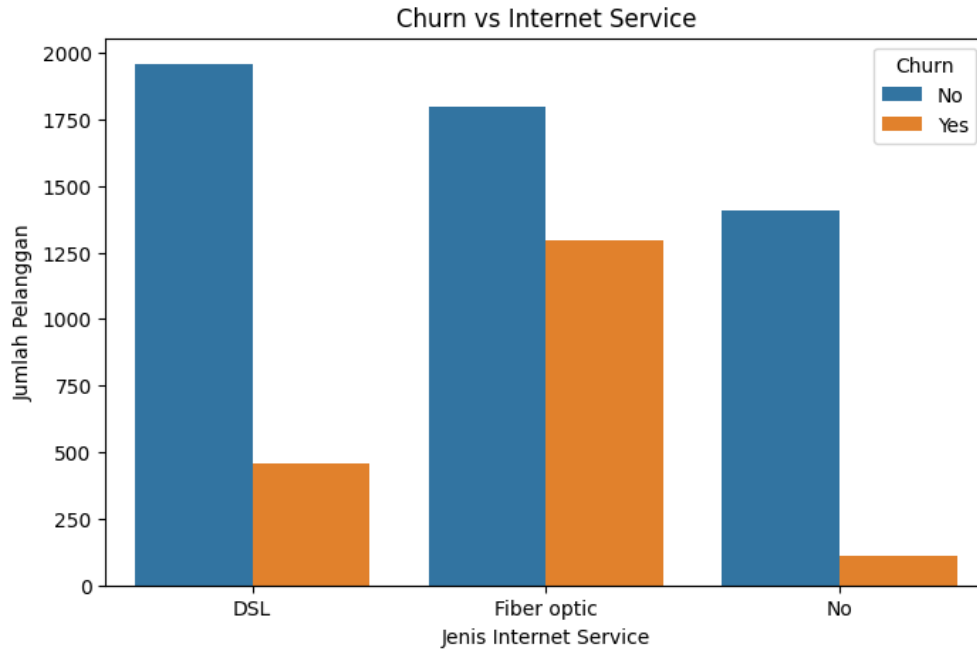
Untuk pertanyaan ketiga yaitu apakah pelanggan yang memilih layanan keamanan online cenderung churn, kita akan membuat bar plot untuk melihat jumlah pelanggan berdasarkan pilihan OnlineSecurity. Kita akan membandingkan jumlah pelanggan yang churn dan tidak churn berdasarkan pilihan OnlineSecurity. Gambar 8 menunjukkan hasil dari visualisasi tersebut.



Gambar 8: Jumlah Pelanggan Churn Berdasarkan Pilihan OnlineSecurity

Dari Gambar 8 dapat dilihat bahwa pelanggan yang memilih untuk menggunakan layanan OnlineSecurity memiliki jumlah churn yang lebih rendah dibandingkan dengan pelanggan yang tidak menggunakan layanan OnlineSecurity. Hal ini menunjukkan bahwa layanan OnlineSecurity dapat membantu mengurangi tingkat churn pelanggan.

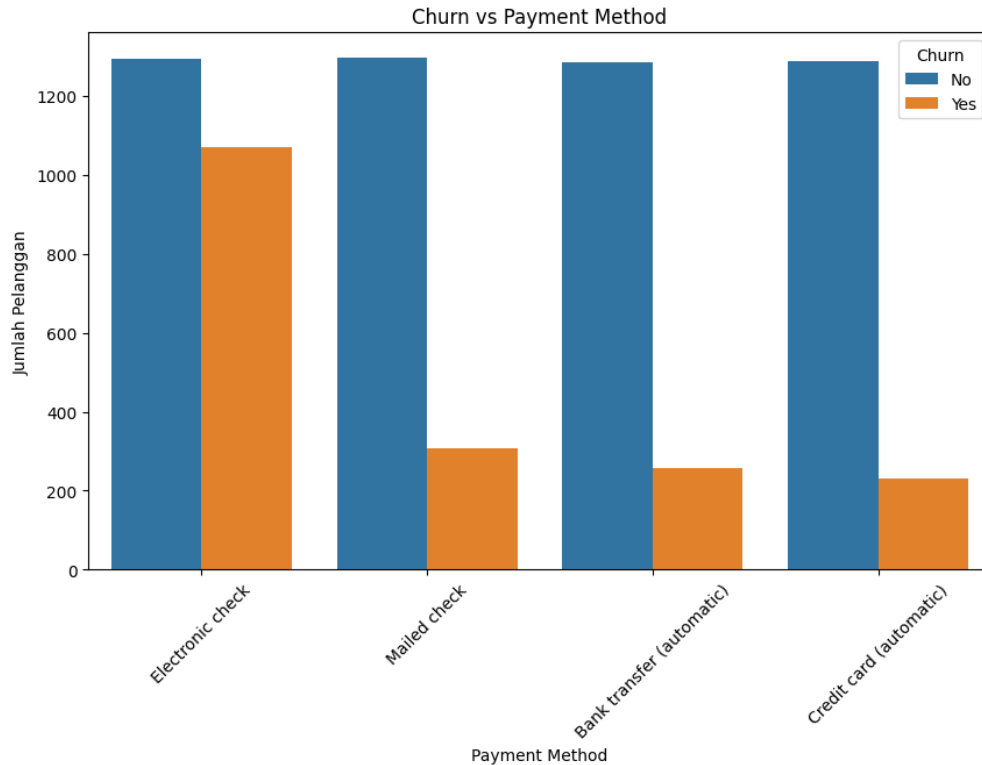
Untuk pertanyaan keempat yaitu apakah pelanggan yang memilih layanan internet tertentu cenderung churn, kita akan membuat bar plot untuk melihat jumlah pelanggan berdasarkan jenis layanan internet. Kita akan membandingkan jumlah pelanggan yang churn dan tidak churn berdasarkan jenis layanan internet. Gambar 9 menunjukkan hasil dari visualisasi tersebut.



Gambar 9: Jumlah Pelanggan Churn Berdasarkan Jenis Layanan Internet

Dari Gambar 9 dapat dilihat bahwa pelanggan yang menggunakan layanan internet "Fiber optic" memiliki jumlah churn yang paling tinggi dibandingkan dengan pelanggan yang menggunakan layanan "DSL" atau "No". Hal ini menunjukkan bahwa pelanggan yang menggunakan layanan internet Fiber optic cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan layanan DSL atau tidak menggunakan layanan internet.

Untuk pertanyaan terakhir yaitu apakah pelanggan yang memilih metode pembayaran tertentu cenderung churn, kita akan membuat bar plot untuk melihat jumlah pelanggan berdasarkan metode pembayaran. Kita akan membandingkan jumlah pelanggan yang churn dan tidak churn berdasarkan metode pembayaran. Gambar 10 menunjukkan hasil dari visualisasi tersebut.



Gambar 10: Jumlah Pelanggan Churn Berdasarkan Metode Pembayaran

Dari Gambar 10 dapat dilihat bahwa pelanggan yang menggunakan metode pembayaran "Electronic check" memiliki jumlah churn yang paling tinggi dibandingkan dengan metode pembayaran lainnya. Hal ini menunjukkan bahwa pelanggan yang menggunakan metode pembayaran Electronic check cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan metode pembayaran lainnya.

Dari hasil analisa diatas kita tidak hanya mendapatkan fitur yang relevan dalam membangun model klasifikasi, tetapi juga mendapatkan insight bisnis yang dapat digunakan untuk meningkatkan retensi pelanggan. Adapun insight yang didapatkan dari analisis fitur kategorikal ini adalah sebagai berikut:

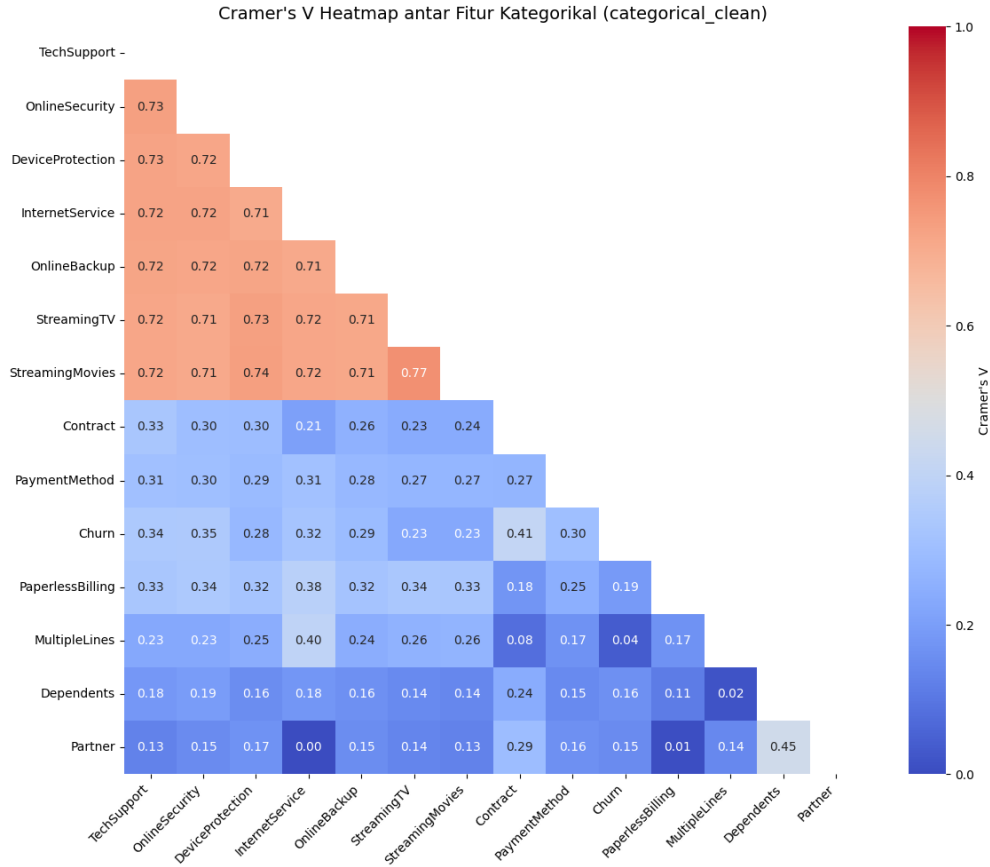
- Pelanggan dengan kontrak "Month-to-month" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang bekerja dengan kontrak tahunan atau dua tahunan.
- Layanan TechSupport dapat membantu mengurangi tingkat churn pelanggan.
- Layanan OnlineSecurity dapat membantu mengurangi tingkat churn pelanggan.
- Pelanggan yang menggunakan layanan internet "Fiber optic" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan layanan DSL atau tidak menggunakan layanan internet.

- Pelanggan yang menggunakan metode pembayaran "Electronic check" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan metode pembayaran lainnya.

Sehingga kita bisa membuat keputusan bisnis yang lebih baik berdasarkan hasil analisis ini. Adapun implikasi bisnisnya dari insight yang dihasilkan adalah sebagai berikut:

- Perusahaan dapat meningkatkan layanan TechSupport untuk membantu mengurangi tingkat churn pelanggan.
- Perusahaan dapat meningkatkan layanan OnlineSecurity untuk membantu mengurangi tingkat churn pelanggan.
- Perusahaan dapat mempertimbangkan untuk meningkatkan kualitas layanan internet Fiber optic untuk mengurangi tingkat churn pelanggan. Atau pun mengurangi biaya langganan untuk pelanggan yang menggunakan layanan internet Fiber optic.
- Perusahaan dapat mempertimbangkan untuk menawarkan metode pembayaran yang lebih fleksibel kepada pelanggan yang menggunakan metode pembayaran Electronic check.
- Perusahaan dapat melakukan penawaran khusus atau promosi untuk pelanggan yang masih bekerja dengan kontrak "Month-to-month" untuk meningkatkan retensi pelanggan.

Selain insight dan implikasi bisnis diatas, kita juga dapat fitur yang relevan untuk digunakan pada feature selection pada model klasifikasi kita nantinya. Namun sebelumnya kita juga perlu tahu korelasi antar fitur kategorikal ini agar kita mengetahui fitur apa saja yang redundant, Adapun kita akan menggunakan Cramer V yang sudah kita dapatkan tadi untuk menghitung korelasi antar fitur kategorikal. Cramer V adalah ukuran asosiasi antara dua variabel kategorikal yang berkisar antara 0 (tidak ada asosiasi) hingga 1 (asosiasi sempurna). Kita akan membuat matriks korelasi menggunakan Cramer V untuk melihat hubungan antar fitur kategorikal. Gambar 11 menunjukkan hasil dari heatmap korelasi Cramer V antar fitur Kategorikal.



Gambar 11: Heatmap Korelasi Cramer V Antar Fitur Kategorikal

Dari Gambar 11 dapat dilihat bahwa terdapat beberapa fitur yang memiliki korelasi tinggi ditandai dengan warna yang lebih merah. Dan fitur-fitur yang memiliki korelasi rendah ditandai dengan warna yang lebih biru. Adapun dengan temuan ini kita bisa melakukan feature selection untuk memilih fitur-fitur yang relevan dan tidak redundant. Kita akan memilih fitur-fitur yang memiliki korelasi tinggi dengan target variabel Churn dan menghindari fitur-fitur yang memiliki korelasi tinggi satu sama lain.

3.3.2 Analisis Fitur Numerik

Analisis fitur numerik dilakukan untuk memahami distribusi dari setiap fitur numerik dalam dataset. Kita akan menggunakan visualisasi seperti histogram, lalu melihat heatmap korelasi untuk melihat hubungan antar fitur numerik. Kita juga akan melakukan analisis statistik untuk melihat apakah terdapat outliers pada fitur-fitur numerik.

Pertama-tama, kita akan melihat ukuran pemusatan dari setiap fitur numerik. Kita akan menggunakan `.describe()` untuk melihat ukuran pemusatan seperti mean, median, dan standar deviasi dari setiap fitur numerik. Dengan begitu kita bisa mendapatkan gambaran umum tentang distribusi dari setiap fitur numerik.

Listing 6: Ukuran Pemusatan Fitur Numerik

```

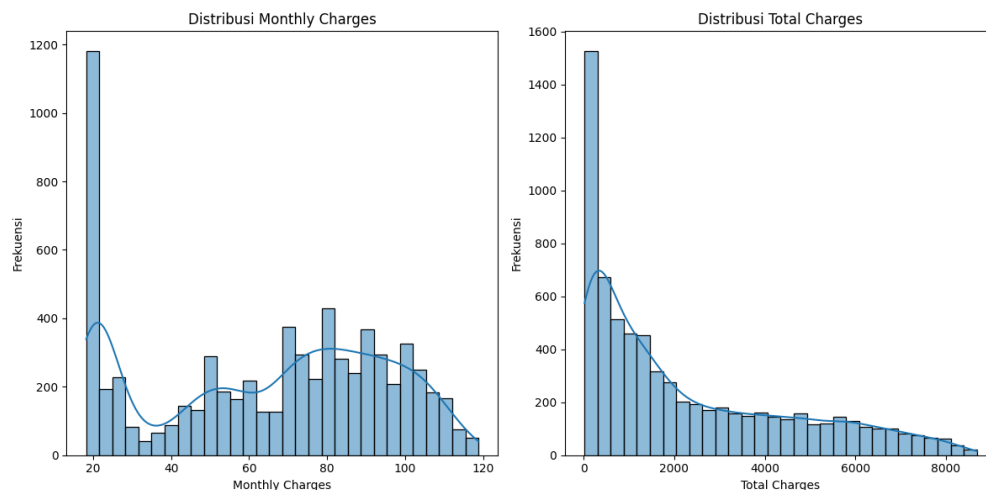
1 # Ukuran pemusatan fitur numerik
2
3 tenure MonthlyCharges TotalCharges
4 count 7032.000000 7032.000000 7032.000000
5 mean 32.421786 64.798208 2283.300441
6 std 24.545260 30.085974 2266.771362
7 min 1.000000 18.250000 18.800000
8 25% 9.000000 35.587500 401.450000
9 50% 29.000000 70.350000 1397.475000
10 75% 55.000000 89.862500 3794.737500
11 max 72.000000 118.750000 8684.800000

```

Dari hasil ukuran pemusatan di atas, kita bisa melihat bahwa fitur "tenure" memiliki nilai rata-rata sekitar 32 bulan, dengan nilai minimum 1 bulan dan maksimum 72 bulan. Fitur "MonthlyCharges" memiliki nilai rata-rata sekitar 64.8, dengan nilai minimum 18.25 dan maksimum 118.75. Sedangkan fitur "TotalCharges" memiliki nilai rata-rata sekitar 2283.3, dengan nilai minimum 18.8 dan maksimum 8684.8.

Setelah di analisa fitur tenure nampaknya cocok untuk dilakukan binning, karena fitur ini merupakan fitur numerik yang memiliki informasi tentang lama berlangganan pelanggan. Kita akan melakukan binning pada fitur "tenure" untuk mengelompokkan pelanggan berdasarkan lama berlangganan mereka. Kita akan membuat 3 kelompok yaitu "0-12 bulan", "13-24 bulan", dan "25+bulan". Dengan begitu kita bisa melihat distribusi pelanggan berdasarkan lama berlangganan mereka.

Karena fitur tenure sudah kita putuskan untuk binning, maka selanjutnya kita akan cek tipe distribusi untuk fitur MonthlyCharges dan TotalCharges. Kita akan menggunakan histogram untuk melihat distribusi dari setiap fitur numerik. Dengan menggunakan .hist() kita bisa membuat histogram yang jelas dan mudah dipahami. Gambar 12 menunjukkan hasil dari histogram distribusi kedua fitur numerik.

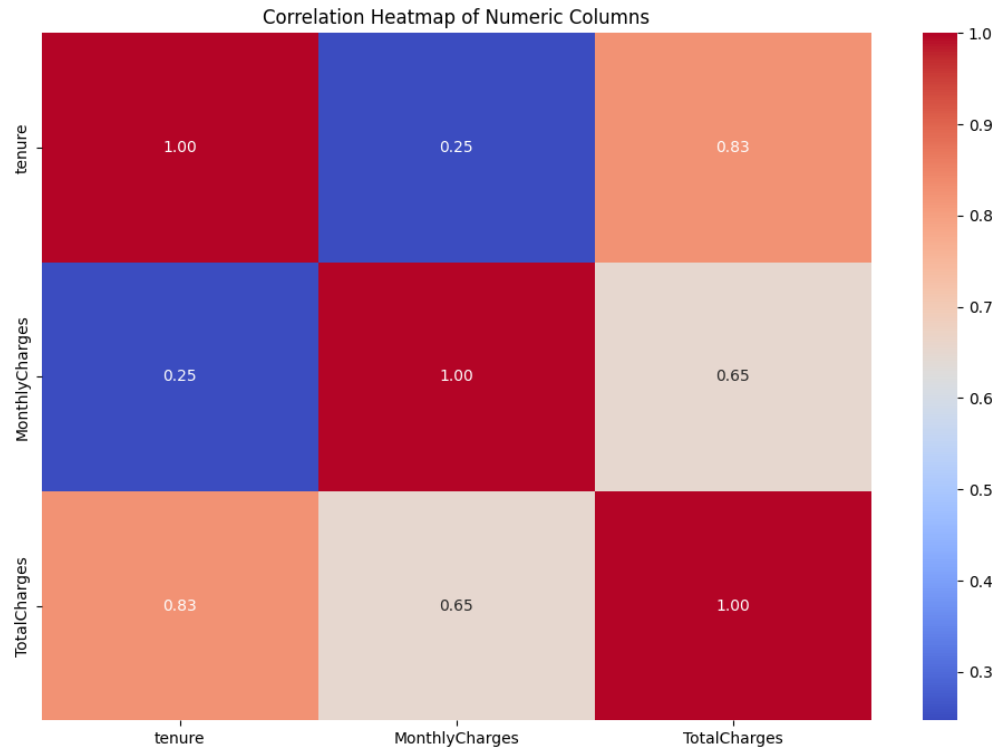


Gambar 12: Histogram Distribusi Fitur Numerik

Dari Gambar 12 dapat dilihat bahwa fitur "MonthlyCharges" memiliki distribusi yang

cenderung normal, dengan puncak di sekitar nilai 70. Sedangkan fitur "TotalCharges" memiliki distribusi yang cenderung skewed ke kanan, dengan puncak di sekitar nilai 1000. Hal ini menunjukkan bahwa sebagian besar pelanggan memiliki total biaya yang relatif rendah, namun terdapat beberapa pelanggan yang memiliki total biaya yang sangat tinggi.

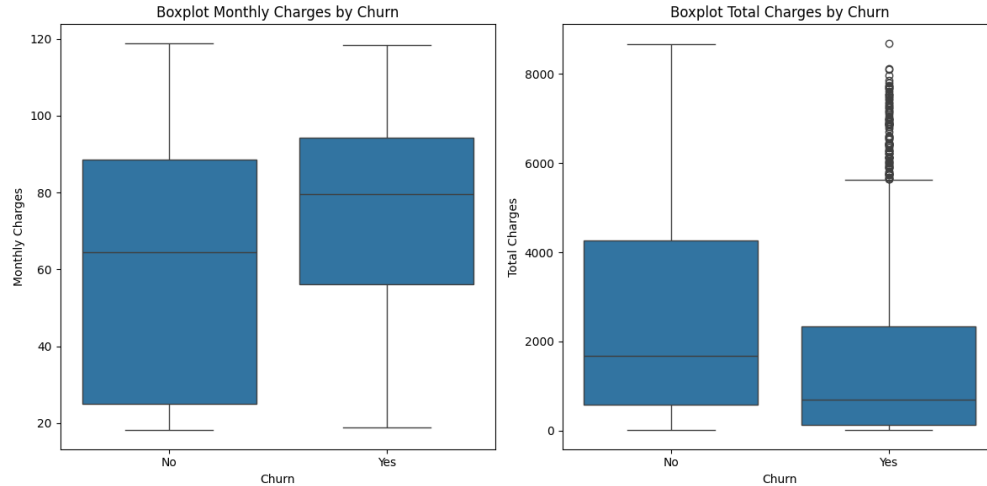
Selanjutnya kita akan melakukan analisis korelasi antar fitur numerik. Kita akan menggunakan heatmap untuk melihat hubungan antar fitur numerik. Dengan menggunakan `.heatmap()` dari Seaborn, kita bisa membuat visualisasi yang jelas dan mudah dipahami. Gambar 13 menunjukkan hasil dari heatmap korelasi antar fitur numerik.



Gambar 13: Heatmap Korelasi Antar Fitur Numerik

Dari Gambar 13 dapat dilihat bahwa terdapat korelasi yang cukup tinggi antara fitur "MonthlyCharges" dan "TotalCharges", dengan nilai korelasi sekitar 0.65. Hal ini menunjukkan bahwa semakin tinggi biaya bulanan yang dibayarkan oleh pelanggan, semakin tinggi pula total biaya yang dibayarkan selama berlangganan. Sedangkan fitur "tenure" memiliki korelasi yang rendah dengan kedua fitur numerik lainnya, yaitu "MonthlyCharges" dan "TotalCharges".

Kita juga akan melakukan analisis outliers pada fitur-fitur numerik. Kita akan menggunakan boxplot untuk melihat apakah terdapat outliers pada setiap fitur numerik. Dengan menggunakan `.boxplot()` dari Seaborn, kita bisa membuat visualisasi yang jelas dan mudah dipahami. Gambar 14 menunjukkan hasil dari boxplot outliers pada kedua fitur numerik.



Gambar 14: Boxplot Outliers Fitur Numerik

Dari Gambar 14 merupakan boxplot untuk fitur "MonthlyCharges" dan "TotalCharges" berdasarkan kategori "Churn". Dapat dilihat bahwa terdapat beberapa outliers pada kedua fitur numerik tersebut. Outliers ini dapat mempengaruhi hasil analisis dan model yang akan dibangun, sehingga perlu ditangani dengan baik. Kita bisa menghapus outliers tersebut atau menggunakan metode robust scaling untuk mengurangi pengaruh outliers pada model.

Selain itu kita juga mendapatkan bahwa pelanggan dengan biaya bulanan yang lebih tinggi cenderung untuk churn. Hal ini terlihat dari boxplot yang menunjukkan bahwa pelanggan yang churn memiliki biaya bulanan yang lebih tinggi dibandingkan dengan pelanggan yang tidak churn. Sedangkan untuk fitur "TotalCharges", kita tidak bisa menarik kesimpulan yang sama karena terdapat outliers yang cukup signifikan pada kedua kategori dan pada pelanggan yang churn berdasarkan totalcharges cukup datanya cukup tersebar sehingga tidak bisa ditarik kesimpulan yang signifikan.

Setelah melakukan analisis fitur numerik, kita mendapatkan beberapa insight yang dapat digunakan untuk meningkatkan retensi pelanggan. Adapun insight yang didapatkan dari analisis fitur numerik ini adalah sebagai berikut:

- Pelanggan dengan lama berlangganan yang lebih lama cenderung memiliki total biaya yang lebih tinggi.
- Pelanggan dengan biaya bulanan yang lebih tinggi cenderung memiliki total biaya yang lebih tinggi.
- Terdapat segmen highend pada fitur TotalCharges menandakan adanya pelanggan yang memiliki total biaya yang sangat tinggi.
- Pelanggan dengan biaya bulanan yang lebih tinggi cenderung untuk churn.

Sehingga kita bisa membuat keputusan bisnis yang lebih baik berdasarkan hasil analisis ini. Adapun implikasi bisnisnya dari insight yang dihasilkan adalah sebagai berikut:

- Perusahaan dapat mempertimbangkan untuk memberikan penawaran khusus atau promosi kepada pelanggan dengan lama berlangganan yang lebih lama untuk meningkatkan retensi pelanggan.
- Perusahaan dapat mempertimbangkan untuk memberikan penawaran khusus atau promosi kepada pelanggan dengan biaya bulanan yang lebih tinggi untuk meningkatkan retensi pelanggan.
- Perusahaan dapat mempertimbangkan untuk melakukan segmentasi pelanggan berdasarkan total biaya yang dibayarkan untuk meningkatkan retensi pelanggan.

Selain kita juga mendapatkan bahwa fitur tenure yang merupakan fitur numerik yang memiliki informasi tentang lama berlangganan pelanggan, dapat dilakukan binning untuk mengelompokkan pelanggan berdasarkan lama berlangganan mereka. Kita akan membuat 3 kelompok yaitu "0-12 bulan", "13-24 bulan", dan "25+bulan". Maka dengan ini kita bisa melakukan fitur engineering untuk membuat fitur baru yaitu "tenure_group" yang merupakan hasil binning dari fitur "tenure". Kita akan menggunakan `pd.cut()` untuk melakukan binning pada fitur "tenure". Dengan begitu kita bisa melihat distribusi pelanggan berdasarkan lama berlangganan mereka.

3.4 Feature Engineering

Setelah melakukan analisis terhadap fitur-fitur yang ada dalam dataset, kita akan melakukan feature engineering untuk membuat fitur-fitur baru yang dapat meningkatkan performa model klasifikasi yang akan dibangun. Feature engineering adalah proses menciptakan fitur-fitur baru dari fitur-fitur yang sudah ada untuk meningkatkan kemampuan model dalam memprediksi target variabel.

3.4.1 Binning Fitur Tenure

Pada analisa sebelumnya kita mendapatkan bahwa fitur tenure yang merupakan fitur numerik yang memiliki informasi tentang lama berlangganan pelanggan, dapat dilakukan binning untuk mengelompokkan pelanggan berdasarkan lama berlangganan mereka. Kita akan membuat 3 kelompok yaitu "0-12 bulan", "13-24 bulan", dan "25+bulan". Maka dengan ini kita bisa melakukan fitur engineering untuk membuat fitur baru yaitu "tenure_group" yang merupakan hasil binning dari fitur "tenure". Kita akan menggunakan `pd.cut()` untuk melakukan binning pada fitur "tenure". Dengan begitu kita bisa melihat distribusi pelanggan berdasarkan lama berlangganan mereka.

Listing 7: Feature Engineering Binning Fitur Tenure

```

1 # Output setelah binning
2 tenure_group
3
4 oneyear
5 threeyear
6 oneyear
7 threeyear

```

8 oneyear

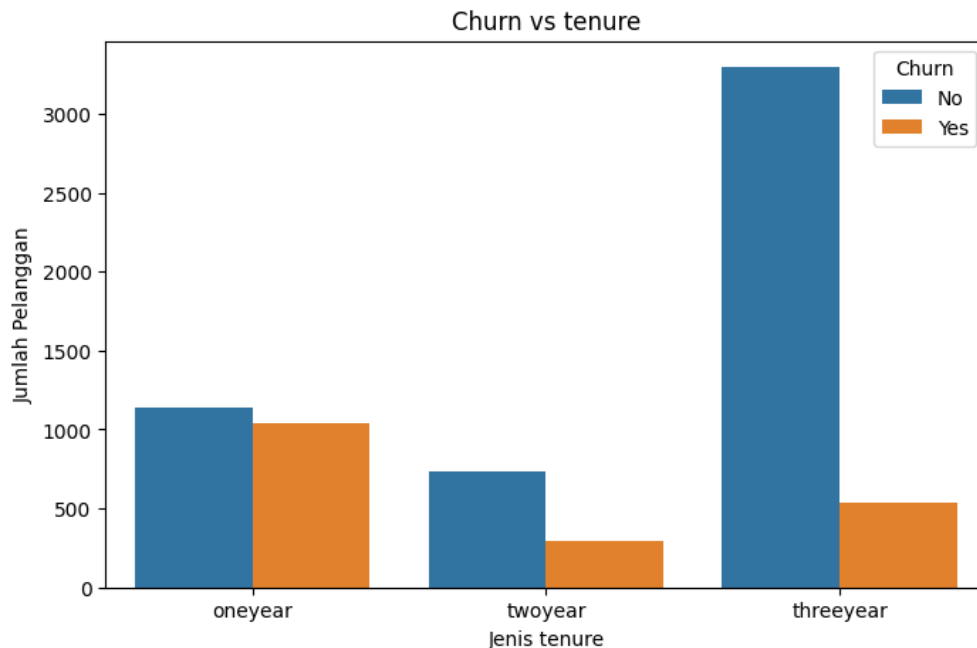
Setelah binning berhasil, selanjutnya saya ingin menguji chi-square untuk melihat apakah fitur tenure_group ini signifikan terhadap target variabel Churn. Dengan begitu kita bisa mengetahui apakah fitur ini relevan untuk digunakan dalam model klasifikasi.

Listing 8: Uji Chi-Square Fitur Tenure Group

```
1 # Output uji chi-square
2 Chi2: 807.5820584283829, p-value: 4.32298939601873e-176, dof: 2
3 Ada hubungan signifikan antara tenure dan Churn
```

Dari hasil uji chi-square di atas, kita mendapatkan nilai Chi2 sebesar 807.58 dengan p-value yang sangat kecil yaitu 4.32e-176. Hal ini menunjukkan bahwa terdapat hubungan yang signifikan antara fitur tenure_group dan target variabel Churn. Sehingga fitur tenure_group ini relevan untuk digunakan dalam model klasifikasi.

Selanjutnya saya juga ingin melihat boxplotnya untuk menggali lebih dalam tentang fitur tenure_group ini. Dengan menggunakan .boxplot() dari Seaborn, kita bisa membuat visualisasi yang jelas dan mudah dipahami. Gambar 15 menunjukkan hasil dari boxplot fitur tenure_group berdasarkan kategori Churn.



Gambar 15: Boxplot Fitur Tenure Group Berdasarkan Kategori Churn

Dari Gambar 15 dapat dilihat bahwa pelanggan yang memiliki lama berlangganan 0-12 bulan cenderung memiliki jumlah churn yang lebih tinggi dibandingkan dengan pelanggan yang memiliki lama berlangganan 13-24 bulan dan 25+bulan. Hal ini menunjukkan bahwa pelanggan yang baru berlangganan cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang sudah lama berlangganan.

3.4.2 Binning fitur dengan Uji Chi-Square lemah

Selain fitur `tenure_group`, kita juga akan melakukan binning pada fitur `Dependents`, `partner`, dan `SeniorCitizen`. Saya memutuskan untuk membuat fitur baru bernama `is_married` dengan kondisi jika `Dependents` atau `Partner` atau `SeniorCitizen` bernilai "Yes" maka `is_married` bernilai "Yes" dan jika tidak maka bernilai "No". Dengan begitu kita bisa melihat distribusi pelanggan berdasarkan status pernikahan mereka.

Listing 9: Feature Engineering Binning

```
1 # Output setelah binning
2
3     is_married
4 1      4313
5 0      2719
6 Name: count, dtype: int64
```

Dengan begitu kita telah berhasil membuat fitur baru bernama `is_married` yang merupakan hasil binning dari fitur `Dependents`, `Partner`, dan `SeniorCitizen`.

3.5 Feature Selection

Setelah melakukan feature engineering, kita akan melakukan feature selection untuk memilih fitur-fitur yang relevan dan tidak redundant. Feature selection adalah proses memilih fitur-fitur yang paling penting untuk digunakan dalam model klasifikasi. Kita akan menggunakan metode chi-square untuk memilih fitur-fitur yang relevan dengan target variabel `Churn`.

Pertama tama `customerID` akan kita buang karena fitur ini tidak relevan untuk digunakan dalam model klasifikasi. Fitur ini hanya berfungsi sebagai identifikasi unik untuk setiap pelanggan dan tidak memberikan informasi yang berguna untuk memprediksi target variabel `Churn`.

3.5.1 Berdasarkan Uji Chi-Square

Kita akan melakukan uji chi-square untuk setiap fitur kategorikal yang ada dalam dataset. Dan telah melakukan feature engineering berdasarkan uji chi-square yang telah dilakukan sebelumnya. Dengan begitu otomatis fitur-fitur ini akan dibuang :

- `Dependents`
- `Partner`
- `SeniorCitizen`
- `tenure`
- `Gender`
- `PhoneService`
- `MultipleLines`

Dengan membuang fitur yang tidak relevan, kita akan mendapatkan fitur-fitur yang lebih fokus dan dapat meningkatkan performa model klasifikasi. Adapun fitur-fitur yang akan digunakan dalam model klasifikasi adalah sebagai berikut:

- tenure_group
- is_married
- InternetService
- OnlineSecurity
- OnlineBackup
- DeviceProtection
- TechSupport
- StreamingTV
- StreamingMovies
- PaperlessBilling
- PaymentMethod
- Contract
- Churn

3.6 Data Preprocessing

Setelah melakukan feature selection, kita akan melakukan data preprocessing untuk mempersiapkan data sebelum digunakan dalam model klasifikasi. Data preprocessing adalah proses membersihkan dan memformat data agar siap digunakan dalam model klasifikasi. Adapun jenis preprocessing yang akan dilakukan akan tergantung pada jenis data yang ada dalam dataset. Kita akan melakukan preprocessing untuk fitur kategorikal.

3.6.1 Preprocessing Fitur Kategorikal

Agar data kategorikal dapat digunakan dalam model klasifikasi, kita perlu mengubah data kategorikal menjadi data numerik. Kita akan menggunakan teknik one-hot encoding untuk mengubah data kategorikal menjadi data numerik. One-hot encoding adalah teknik yang mengubah setiap kategori menjadi kolom baru dengan nilai 0 atau 1. Adapun fitur yang cocok untuk dilakukan one-hot encoding adalah fitur kategorikal yang memiliki lebih dari 2 kategori. Kita akan menggunakan `pd.get_dummies()` untuk melakukan one-hot encoding pada fitur kategorikal.

Listing 10: Preprocessing Fitur Kategorikal

```

1 # Preprocessing fitur kategorikal setelah data di split
2
3 X_train = pd.get_dummies(X_train, columns=['tenure_group', 'is_
4 married', 'InternetService', 'OnlineSecurity',
5 'OnlineBackup', 'DeviceProtection', 'TechSupport',
6 'StreamingTV', 'StreamingMovies', 'PaperlessBill
7 ing', 'PaymentMethod', 'Contract'], drop_first=True)
8
9 X_test = pd.get_dummies(X_test, columns=['tenure_group',
10 'is_married', 'InternetService', 'OnlineSecurity',
11 'OnlineBackup', 'DeviceProtection', 'TechSupport',
12 'StreamingTV', 'StreamingMovies', 'PaperlessBilling',
13 'PaymentMethod', 'Contract'], drop_first=True)
14
15 # Menyelaraskan kolom pada X_test dengan X_train
16 X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

```

Setelah melakukan one-hot encoding, kita akan mendapatkan fitur-fitur baru yang merupakan hasil dari one-hot encoding. Adapun untuk fitur-fitur yang hanya terdiri dari 2 kategori seperti `is_married`, kita akan mengubahnya menjadi 0 dan 1. Fitur ini akan diubah menjadi numerik dengan cara mengubah nilai "Yes" menjadi 1 dan "No" menjadi 0. Dengan begitu kita akan mendapatkan fitur-fitur yang siap digunakan dalam model klasifikasi. Gambar 16 menunjukkan hasil dari preprocessing pada fitur kategorikal.

1,78,0,1,0,1,0,1,0,0,0,0,1,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0
0,20,0,3,0,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,1,0,0,0,1
0,20,0,1,0,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,0,1
1,105,1,3,0,1,0,1,0,0,0,0,1,0,0,1,1,0,0,0,0,1,0,0,1,0,0,1,1,0,0,0
1,105,0,3,0,1,0,1,0,0,0,0,1,0,0,1,1,0,0,0,0,1,0,0,1,1,0,0,0,1,0
0,24,1,3,0,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0
0,20,0,1,0,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,1
1,87,1,2,0,1,0,1,0,0,1,0,0,0,0,1,1,0,0,1,0,0,0,0,1,1,0,0,0,1,0
1,61,0,3,1,0,0,0,0,1,1,0,0,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,1,0,0
1,70,1,1,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,1,0
0,19,0,1,0,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,1,0,0
1,78,1,2,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,1,1,0,0,0,1,0
1,40,1,3,1,0,0,0,0,1,1,0,0,0,0,1,0,0,1,1,0,0,1,0,0,0,1,0,0,1,0
0,51,1,3,1,0,0,1,0,0,1,0,0,1,0,0,0,0,1,0,0,1,0,0,1,1,0,0,0,1,0,0
1,90,0,1,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,1,0,0,1,1,0,0,0,1,0
0,75,0,1,0,1,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,0,0,1
1,79,0,2,0,1,0,1,0,0,0,0,1,1,0,0,1,0,0,1,0,0,1,0,0,1,0,0,0,1,0

Gambar 16: Hasil One-Hot Encoding Fitur Kategorikal

Setelah melakukan preprocessing pada fitur kategorikal, kita akan mendapatkan data yang siap digunakan dalam model klasifikasi. Namun sebelum itu kita juga perlu melakukan preprocessing pada fitur numerik.

3.6.2 Preprocessing Fitur Numerik

Untuk fitur numerik, kita hanya memiliki 1 fitur yang perlu dilakukan preprocessing yaitu fitur `MonthlyCharges`. Namun karena ini adalah satu satunya fitur numerik yang ada dalam dataset, saya memutuskan untuk tidak melakukan preprocessing pada fitur ini. Kita akan menggunakan fitur ini apa adanya tanpa melakukan scaling atau normalisasi. Hal ini karena fitur ini merupakan satu satunya fitur numerik sehingga tidak ada fitur lain yang perlu diselaraskan dengan fitur ini.

Selain itu kita juga tidak perlu melakukan transformasi log pada fitur ini karena fitur ini sudah memiliki distribusi yang cenderung normal. Sehingga kita akan menggunakan fitur `MonthlyCharges` apa adanya tanpa melakukan preprocessing.

Setelah semua siap kita akan lanjut ke tahap selanjutnya yaitu membangun model klasifikasi. Kita akan menggunakan model klasifikasi yang sudah umum digunakan seperti Logistic Regression, Decision Tree, Random Forest, dan XGBoost. Kita akan membandingkan performa dari setiap model untuk memilih model yang terbaik.

3.7 Handling Imbalanced Data

Setelah melakukan preprocessing pada data, kita akan melakukan handling imbalanced data. Imbalanced data adalah kondisi di mana jumlah data pada setiap kelas tidak seimbang. Hal ini dapat mempengaruhi performa model klasifikasi yang akan dibangun. Namun pertama tama kita akan melakukan analisis terhadap distribusi kelas pada target variabel `Churn`. Kita akan menggunakan `.value_counts()` untuk melihat distribusi kelas pada target variabel `Churn`.

Listing 11: Distribusi Kelas pada Target Variabel `Churn`

```
1 # Distribusi kelas pada target variabel Churn
2 Kelas 0: 4130 sampel
3 Kelas 1: 1495 sampel
```

Dari hasil distribusi kelas pada target variabel `Churn`, kita mendapatkan bahwa jumlah sampel pada kelas 0 (tidak churn) jauh lebih banyak dibandingkan dengan kelas 1 (churn). Hal ini menunjukkan bahwa data yang kita miliki tidak seimbang.

Namun jika dihitung dari rasionya jumlah kelas 0 (tidak churn) adalah sekitar 73.8% dan jumlah kelas 1 (churn) adalah sekitar 26.2%. Hal ini menunjukkan bahwa data yang kita miliki masih dalam batas wajar untuk digunakan dalam model klasifikasi. Sehingga kita tetap perlu melakukan tuning pada model dengan menambahkan `class_weight='balanced'` pada setiap model yang akan kita gunakan. Dengan begitu model akan memberikan bobot yang lebih besar pada kelas yang kurang terwakili yaitu kelas 1 (churn).

4 Pengujian dan analisis

Pada bab ini, akan dijelaskan mengenai hasil pengujian dan pembahasan dari penelitian yang telah diuraikan pada metodologi. Selain itu, akan dipaparkan juga mengenai skenario pengujian yang dilakukan untuk mengevaluasi performa sistem secara keseluruhan. Pengujian ini dilakukan dengan tujuan untuk memastikan bahwa sistem yang dirancang mampu berfungsi dengan baik dalam memprediksi apakah pelanggan akan churn atau tidak berdasarkan data yang telah diproses pada tahap sebelumnya.

4.1 Pengujian Sistem

Pengujian sistem dilakukan dengan menggunakan dataset yang telah diolah sebelumnya dan dibagi menjadi data latih dan data uji. Data latih digunakan untuk melatih model Logistic Regression, sedangkan data uji digunakan untuk menguji performa model yang telah dilatih. Adapun skenario pengujian yang dilakukan adalah sebagai berikut:

1. **Pengujian Model Logistic Regression:** Model Logistic Regression dilatih menggunakan data latih yang telah disiapkan. Proses pelatihan ini melibatkan penyesuaian parameter model untuk memaksimalkan akurasi prediksi terhadap data latih.
2. **Perbandingan dengan Model Lain:** Jika ada, model lain yang lebih kompleks seperti Decision Tree, Random Forest atau XGBoost juga diuji untuk membandingkan performa dengan model Logistic Regression. Hal ini dilakukan untuk melihat apakah model yang lebih kompleks memberikan hasil yang lebih baik dalam memprediksi churn pelanggan.

4.2 Pengujian model Logistic Regression

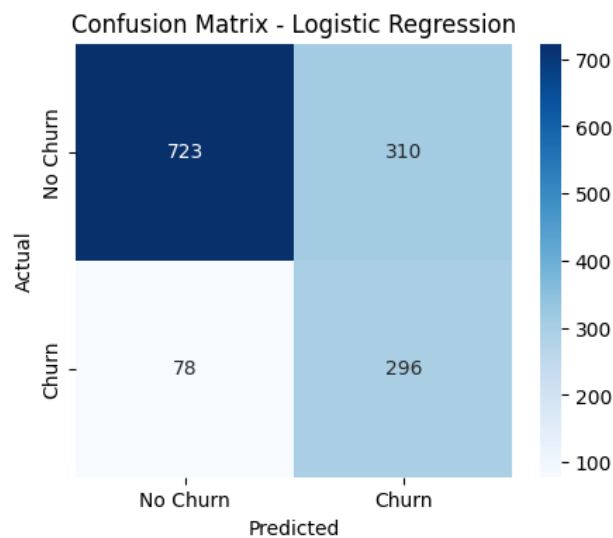
Pengujian model Logistic Regression dilakukan dengan menggunakan data uji yang telah disiapkan. Model ini dilatih pada data latih dan kemudian diuji pada data uji untuk mengevaluasi performanya. Hasil dari pengujian ini akan memberikan gambaran tentang seberapa baik model dalam memprediksi churn pelanggan.

Setelah dilakukan pengujian model Logistic Regression, diperoleh hasil confusion matrix yang merepresentasikan performa model dalam mengklasifikasikan data uji. Confusion matrix ini menunjukkan jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Dari confusion matrix, dapat dihitung beberapa metrik evaluasi seperti akurasi, presisi, recall, dan F1-score.

4.2.1 Hasil Logistic Regression tanpa hyperparameter tuning

Berikut adalah hasil confusion matrix dari model Logistic Regression yang telah diuji pada data uji, Gambar 17 menunjukkan confusion matrix yang diperoleh dari model Logistic Regression tanpa melakukan hyperparameter tuning. Sehingga baseline model Logistic Regression ini akan menjadi acuan untuk perbandingan dengan model-model lainnya yang akan diuji pada tahap selanjutnya. Namun kita tetap perlu menambahkan beberapa setting manual karena pada bab sebelumnya kita menemukan imbalance data pada dataset yang digunakan. Oleh karena itu, kita perlu mengatur beberapa parameter pada model Logistic Regression untuk mengatasi masalah tersebut. Berikut adalah parameter yang digunakan pada model Logistic Regression:

- **max_iter=1000**: Menentukan jumlah iterasi maksimum pada proses optimisasi. Nilai ini dipilih agar proses pelatihan memiliki cukup iterasi untuk mencapai konvergensi.
- **class_weight='balanced'**: Mengatur bobot kelas secara otomatis berdasarkan proporsi jumlah sampel pada setiap kelas. Parameter ini digunakan untuk mengatasi permasalahan ketidakseimbangan data (*imbalanced dataset*) dengan memberikan bobot lebih besar pada kelas minoritas (Churn).
- **solver='liblinear'**: Algoritma optimisasi yang digunakan. *Liblinear* cocok untuk dataset berukuran kecil hingga menengah dan mendukung regularisasi L1 maupun L2.



Gambar 17: Confusion Matrix Model Logistic Regression

Dari gambar 17 kita bisa identifikasi performa model Logistic Regression dalam mengklasifikasikan data uji. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 723 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)
- False Positive (FP): 310 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 78 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 296 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Logistic Regression. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Logistic Regression:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{296 + 723}{296 + 723 + 310 + 78} = \frac{1019}{1407} \approx 0.724 \quad (11)$$

Akurasi model Logistic Regression adalah sekitar 72.4%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 72.4% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Logistic Regression:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{296}{296 + 310} = \frac{296}{606} \approx 0.488 \quad (12)$$

Presisi model Logistic Regression adalah sekitar 48.8%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 48.8% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Logistic Regression:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{296}{296 + 78} = \frac{296}{374} \approx 0.791 \quad (13)$$

Recall model Logistic Regression adalah sekitar 79.1%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 79.1% dari pelanggan yang sebenarnya churn, namun masih ada 20.9% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Logistic Regression:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.488 \cdot 0.791}{0.488 + 0.791} \approx 0.604 \quad (14)$$

F1-score model Logistic Regression adalah sekitar 60.4%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Logistic Regression adalah 0.8279. Nilai ini menunjukkan bahwa model memiliki kemampuan yang baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.8279, model Logistic Regression ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.2.2 Interpretasi Hasil dan Implikasi Bisnis

Hasil pengujian model Logistic Regression menunjukkan bahwa model ini memiliki akurasi sekitar 72.4%, dengan presisi 48.8%, recall 79.1%, dan F1-score 60.4%. Meskipun model ini mampu menangkap sebagian besar pelanggan yang churn (recall yang tinggi), namun presisi yang rendah menunjukkan bahwa model ini juga menghasilkan banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak yang mungkin akan mengakibatkan perusahaan mengeluarkan biaya tambahan untuk intervensi yang tidak perlu, seperti penawaran khusus atau peningkatan layanan kepada pelanggan yang sebenarnya tidak berisiko churn.

Menurut saya metrik evaluasi yang paling relevan untuk kasus ini adalah recall, karena dalam konteks bisnis, penting untuk menangkap sebanyak mungkin pelanggan yang berpotensi churn. Dengan recall yang tinggi, perusahaan dapat lebih proaktif dalam melakukan intervensi untuk mencegah churn, seperti memberikan penawaran khusus atau meningkatkan kualitas layanan.

Namun, perlu diingat bahwa model ini masih memiliki ruang untuk perbaikan, terutama dalam hal presisi. Perusahaan perlu mempertimbangkan untuk melakukan hyperparameter tuning atau mencoba model lain yang lebih kompleks untuk meningkatkan performa prediksi churn. Pada tahap selanjutnya, akan dilakukan hyperparameter tuning untuk mencoba meningkatkan presisi tanpa mengorbankan recall yang tinggi.

4.2.3 Hasil Logistic Regression dengan hyperparameter tuning

Berdasarkan hasil model baseline Logistic Regression, diketahui bahwa model telah memiliki nilai *recall* yang cukup baik namun masih memiliki ruang perbaikan pada metrik *precision*. Oleh karena itu, dilakukan proses optimisasi hyperparameter untuk mencari kombinasi parameter yang lebih optimal. Metode yang digunakan adalah *RandomizedSearchCV* dengan skema *Stratified K-Fold Cross-Validation* untuk memastikan proporsi kelas tetap seimbang pada setiap fold. Hyperparameter yang diujikan meliputi *c*, *penalty*, *solver*, dan *class_weight*. Kriteria pemilihan model terbaik ditentukan berdasarkan metrik *recall* untuk memaksimalkan kemampuan model dalam mendeteksi pelanggan yang berpotensi churn.

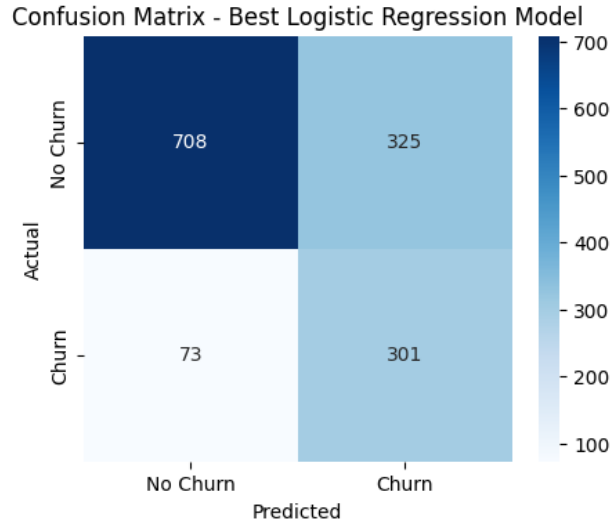
Pemilihan kombinasi terbaik dilakukan berdasarkan metrik *recall*, mengingat tujuan utama dalam kasus ini adalah memaksimalkan deteksi pelanggan churn. Adapun ruang pencarian hyperparameter yang digunakan ditunjukkan pada Tabel 2.

Table 2: Ruang Pencarian Hyperparameter Logistic Regression

Hyperparameter	Nilai yang diuji
Solver	liblinear, saga
Penalty	l1, l2
Regularisasi C	Distribusi log-uniform $[10^{-3}, 10^2]$
Class Weight	None, balanced, {0:1, 1:2}, {0:1, 1:3}

Proses tuning dilakukan menggunakan $n_iter = 50$, yang berarti *RandomizedSearchCV* akan menguji 50 kombinasi hyperparameter secara acak dari ruang pencarian yang ada. Evaluasi dilakukan menggunakan skema *Stratified K-Fold Cross-Validation* untuk mempertahankan proporsi kelas pada setiap fold. Skor yang dihitung meliputi *recall*, *precision*, *f1-score*, dan *ROC-AUC*, dengan metrik *recall* sebagai acuan utama untuk pemilihan model terbaik.

Setelah dilakukan *RandomizedSearchCV*, diperoleh confusion matrix dari model Logistic Regression yang telah dioptimasi hyperparameternya, seperti pada Gambar 18. Confusion matrix ini menunjukkan performa model dalam mengklasifikasikan data uji setelah proses tuning.



Gambar 18: Confusion Matrix Model Logistic Regression dengan Hyperparameter Tuning

Dari gambar 18, kita dapat mengidentifikasi performa model Logistic Regression yang telah dioptimasi hyperparameter. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 708
- False Positive (FP): 325
- False Negative (FN): 73
- True Positive (TP): 301

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Logistic Regression yang telah dioptimasi hyperparameter. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Logistic Regression setelah hyperparameter tuning:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{301 + 708}{301 + 708 + 325 + 73} = \frac{1009}{1407} \approx 0.717 \quad (15)$$

Akurasi model Logistic Regression setelah hyperparameter tuning adalah sekitar 71.7%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 71.7% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Logistic Regression setelah hyperparameter tuning:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{301}{301 + 325} = \frac{301}{626} \approx 0.481 \quad (16)$$

Presisi model Logistic Regression setelah hyperparameter tuning adalah sekitar 48.1%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 48.1% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Logistic Regression setelah hyperparameter tuning:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{301}{301 + 73} = \frac{301}{374} \approx 0.804 \quad (17)$$

Recall model Logistic Regression setelah hyperparameter tuning adalah sekitar 80.4%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 80.4% dari pelanggan yang sebenarnya churn, namun masih ada 19.6% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Logistic Regression setelah hyperparameter tuning:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.481 \cdot 0.804}{0.481 + 0.804} \approx 0.604 \quad (18)$$

F1-score model Logistic Regression setelah hyperparameter tuning adalah sekitar 60.4%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Logistic Regression setelah hyperparameter tuning adalah 0.8279. Nilai ini menunjukkan bahwa model memiliki kemampuan yang baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.8279, model Logistic Regression ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.2.4 Parameter Terbaik

Nilai nilai diatas diperoleh dari proses *RandomizedSearchCV* yang telah dilakukan sebelumnya. Adapun kombinasi hyperparameter untuk mendapatkan hasil diatas ditunjukkan pada Tabel 3. Kombinasi ini merupakan hasil terbaik yang diperoleh dari proses tuning hyperparameter, yang bertujuan untuk meningkatkan performa model dalam mendeteksi pelanggan yang churn.

Table 3: Kombinasi Hyperparameter Terbaik (RandomizedSearchCV)

Hyperparameter	Nilai Terbaik
C	40.679
class_weight	{0: 1, 1: 3}
penalty	12
solver	liblinear

4.2.5 Perbandingan Hasil Logistic Regression Sebelum dan Sesudah Hyperparameter Tuning

Setelah melakukan hyperparameter tuning, kita dapat membandingkan hasil model Logistic Regression sebelum dan sesudah tuning. Tabel 4 menunjukkan perbandingan metrik evaluasi antara model baseline dan model yang telah dioptimasi hyperparameter.

Table 4: Perbandingan Hasil Model Logistic Regression Sebelum dan Sesudah Hyperparameter Tuning

Metrik	Sebelum Tuning	Setelah Tuning
Akurasi	72.4%	71.7%
Presisi	48.8%	48.1%
Recall	79.1%	80.4%
F1-score	60.4%	60.4%
ROC-AUC	0.8279	0.8279

Dari tabel 4, dapat dilihat bahwa setelah melakukan hyperparameter tuning, model Logistic Regression mengalami peningkatan pada metrik recall, yaitu dari 79.1% menjadi 80.4%. Hal ini menunjukkan bahwa model yang telah dioptimasi lebih baik dalam mendeteksi pelanggan yang berpotensi churn.

Namun, akurasi dan presisi model mengalami penurunan, yaitu dari 72.4% menjadi 71.7% untuk akurasi dan dari 48.8% menjadi 48.1% untuk presisi. ROC-AUC tetap sama pada nilai 0.8279, yang menunjukkan bahwa tuning yang dilakukan fokus pada peningkatan recall tanpa mengorbankan kemampuan model dalam membedakan antara kelas churn dan tidak churn.

Meskipun demikian, peningkatan recall yang signifikan menunjukkan bahwa model ini lebih efektif dalam menangkap pelanggan churn, yang merupakan tujuan utama dari analisis ini.

4.3 Pengujian Model Decision Tree

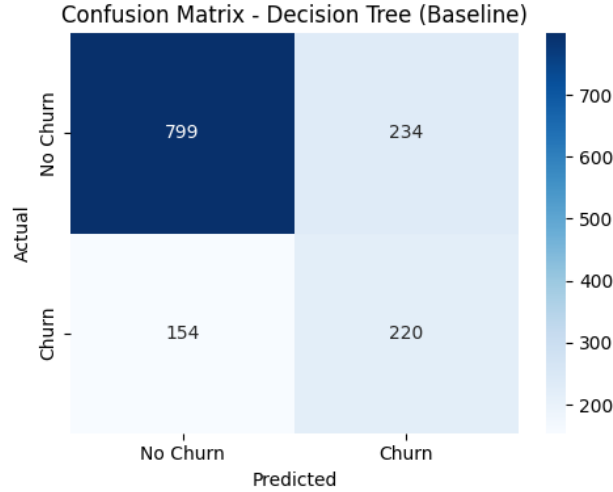
Pengujian model Decision Tree dilakukan dengan menggunakan data uji yang telah disiapkan. Model ini dilatih pada data latih dan kemudian diuji pada data uji untuk mengevaluasi performanya. Hasil dari pengujian ini akan memberikan gambaran tentang seberapa baik model dalam memprediksi churn pelanggan.

Setelah dilakukan pengujian model Decision Tree, diperoleh hasil confusion matrix yang merepresentasikan performa model dalam mengklasifikasikan data uji. Confusion matrix ini menunjukkan jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Dari confusion matrix, dapat dihitung beberapa metrik evaluasi seperti akurasi, presisi, recall, dan F1-score.

4.3.1 Hasil Decision Tree tanpa hyperparameter tuning

Berikut adalah hasil confusion matrix dari model Decision Tree yang telah diuji pada data uji, Gambar 19 menunjukkan confusion matrix yang diperoleh dari model Decision Tree tanpa melakukan hyperparameter tuning. Sehingga baseline model Decision Tree ini akan menjadi acuan untuk perbandingan dengan model-model lainnya yang akan diuji pada tahap selanjutnya. Namun kita tetap perlu menambahkan beberapa setting manual karena pada bab sebelumnya kita menemukan imbalance data pada dataset yang digunakan. Oleh karena itu, kita perlu mengatur beberapa parameter pada model Decision Tree untuk mengatasi masalah tersebut. Parameter yang digunakan adalah sebagai berikut:

- **criterion="gini"**: Fungsi pengukuran impurity yang digunakan adalah Gini Impurity (default). Alternatif lainnya adalah **"entropy"** yang menggunakan Information Gain.
- **splitter="best"**: Strategi pemilihan split pada setiap node adalah memilih split terbaik (default). Alternatifnya adalah **"random"** untuk pemilihan acak.
- **max_depth=None**: Kedalaman pohon tidak dibatasi (default), sehingga pohon akan tumbuh hingga semua daun bersifat *pure* atau tidak dapat di-split lagi. Hal ini berpotensi menimbulkan *overfitting*.
- **min_samples_split=2**: Minimal jumlah sampel untuk melakukan split pada sebuah node adalah 2 (default).
- **min_samples_leaf=1**: Minimal jumlah sampel pada sebuah daun adalah 1 (default).
- **max_features=None**: Semua fitur digunakan untuk mencari split terbaik (default).
- **class_weight="balanced"**: Memberikan bobot kelas secara otomatis yang proporsional terhadap kebalikannya dari frekuensi kelas, untuk mengatasi ketidakseimbangan kelas pada dataset.
- **random_state=42**: Menetapkan nilai *seed* untuk memastikan hasil dapat direproduksi.



Gambar 19: Confusion Matrix Model Decision Tree

Dari gambar 19 kita bisa identifikasi performa model Decision Tree dalam mengklasifikasi data uji. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 799 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)
- False Positive (FP): 234 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 154 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 220 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Decision Tree. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Decision Tree:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{220 + 799}{220 + 799 + 234 + 154} = \frac{1019}{1407} \approx 0.724 \quad (19)$$

Akurasi model Decision Tree adalah sekitar 72.4%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 72.4% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Decision Tree:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{220}{220 + 234} = \frac{220}{454} \approx 0.485 \quad (20)$$

Presisi model Decision Tree adalah sekitar 48.5%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 48.5% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Decision Tree:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{220}{220 + 154} = \frac{220}{374} \approx 0.588 \quad (21)$$

Recall model Decision Tree adalah sekitar 58.8%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 58.8% dari pelanggan yang sebenarnya churn, namun masih ada 41.2% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Decision Tree:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.485 \cdot 0.588}{0.485 + 0.588} \approx 0.531 \quad (22)$$

F1-score model Decision Tree adalah sekitar 53.1%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Decision Tree adalah 0.6866. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.6866, model Decision Tree ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.3.2 Hasil Decision Tree dengan hyperparameter tuning

Setelah memperoleh hasil dari model Decision Tree tanpa hyperparameter tuning, langkah selanjutnya adalah melakukan *hyperparameter tuning* untuk meningkatkan performa model, khususnya dalam hal *recall* yang menjadi fokus utama pada kasus prediksi *churn*. Proses *tuning* dilakukan menggunakan `RandomizedSearchCV` dengan skema *cross-validation*, di mana metrik utama yang digunakan sebagai acuan pemilihan model terbaik adalah *recall*.

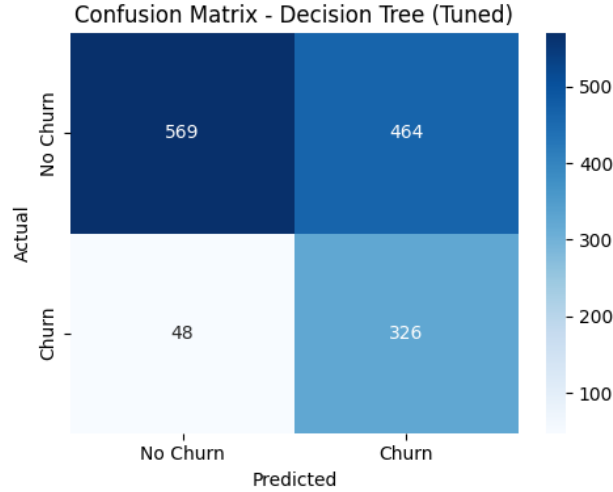
Ruang pencarian (*search space*) untuk parameter model Decision Tree dirancang dengan mempertimbangkan keseimbangan antara kompleksitas model dan risiko *overfitting*, yaitu sebagai berikut:

Table 5: Ruang Pencarian Hyperparameter Decision Tree

Hyperparameter	Nilai yang diuji
Criterion	"gini", "entropy"
Max Depth	3, 4, ..., 30
Min Samples Split	2, 3, ..., 50
Min Samples Leaf	1, 2, ..., 30
Max Features	None, "sqrt", "log2"
Class Weight	None, "balanced"
Ccp Alpha	[0.0, 0.02]
Splitter	"best"

Dari tabel diatas 80 iterasi `RandomizedSearchCV` dilakukan untuk mencari kombinasi hyperparameter yang optimal. Proses ini bertujuan untuk menemukan konfigurasi yang memberikan nilai *recall* tertinggi, sehingga model dapat lebih efektif dalam mendeteksi pelanggan yang berpotensi churn.

Setelah dilakukan `RandomizedSearchCV`, diperoleh confusion matrix dari model Decision Tree yang telah dioptimasi hyperparameternya, seperti pada Gambar 20. Confusion matrix ini menunjukkan performa model dalam mengklasifikasikan data uji setelah proses tuning.



Gambar 20: Confusion Matrix Model Decision Tree dengan Hyperparameter Tuning

Dari gambar 20, kita dapat mengidentifikasi performa model Decision Tree yang telah dioptimasi hyperparameter. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 569 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)
- False Positive (FP): 464 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 48 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 326 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Decision Tree yang telah dioptimasi hyperparameter. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Decision Tree setelah hyperparameter tuning:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{326 + 569}{326 + 569 + 464 + 48} = \frac{895}{1407} \approx 0.636 \quad (23)$$

Akurasi model Decision Tree setelah hyperparameter tuning adalah sekitar 63.6%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 63.6% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Decision Tree setelah hyperparameter tuning:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{326}{326 + 464} = \frac{326}{790} \approx 0.413 \quad (24)$$

Presisi model Decision Tree setelah hyperparameter tuning adalah sekitar 41.3%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 41.3% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Decision Tree setelah hyperparameter tuning:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{326}{326 + 48} = \frac{326}{374} \approx 0.872 \quad (25)$$

Recall model Decision Tree setelah hyperparameter tuning adalah sekitar 87.2%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 87.2% dari pelanggan yang sebenarnya churn, namun masih ada 12.8% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Decision Tree setelah hyperparameter tuning:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.413 \cdot 0.872}{0.413 + 0.872} \approx 0.561 \quad (26)$$

F1-score model Decision Tree setelah hyperparameter tuning adalah sekitar 56.1%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Decision Tree setelah hyperparameter tuning adalah 0.7880. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna.

Dengan nilai 0.7880, model Decision Tree ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.3.3 Parameter Terbaik

Nilai-nilai di atas diperoleh dari proses `RandomizedSearchCV` yang telah dilakukan sebelumnya. Adapun kombinasi hyperparameter untuk mendapatkan hasil di atas ditunjukkan pada Tabel 6. Kombinasi ini merupakan hasil terbaik yang diperoleh dari proses tuning hyperparameter, yang bertujuan untuk meningkatkan performa model dalam mendeteksi pelanggan yang churn.

Table 6: Kombinasi Hyperparameter Terbaik (`RandomizedSearchCV`)

Hyperparameter	Nilai Terbaik
Criterion	log_loss
Max Depth	22
Min Samples Split	38
Min Samples Leaf	3
Max Features	None
Class Weight	balanced
Ccp Alpha	0.0123677
Splitter	best

4.3.4 Perbandingan Hasil Decision Tree Sebelum dan Sesudah Hyperparameter Tuning

Setelah melakukan hyperparameter tuning, kita dapat membandingkan hasil model Decision Tree sebelum dan sesudah tuning. Tabel 7 menunjukkan perbandingan metrik evaluasi antara model baseline dan model yang telah dioptimasi hyperparameter.

Table 7: Perbandingan Hasil Model Decision Tree Sebelum dan Sesudah Hyperparameter Tuning

Metrik	Sebelum Tuning	Setelah Tuning
Akurasi	72.4%	63.6%
Presisi	48.5%	41.3%
Recall	58.8%	87.2%
F1-score	53.1%	56.1%
ROC-AUC	0.6866	0.7880

Dari tabel 7, dapat dilihat bahwa setelah melakukan hyperparameter tuning, model Decision Tree mengalami peningkatan yang signifikan pada metrik recall, yaitu dari 58.8% menjadi 87.2%. Hal ini menunjukkan bahwa model yang telah dioptimasi lebih baik dalam mendeteksi pelanggan yang berpotensi churn.

Namun, akurasi dan presisi model mengalami penurunan, yaitu dari 72.4% menjadi 63.6% untuk akurasi dan dari 48.5% menjadi 41.3% untuk presisi. Meskipun demikian, peningkatan

recall yang signifikan menunjukkan bahwa model ini lebih efektif dalam menangkap pelanggan churn, yang merupakan tujuan utama dari analisis ini.

Meskipun akurasi menurun, peningkatan recall menunjukkan bahwa model ini lebih fokus pada deteksi pelanggan churn, yang merupakan prioritas utama dalam konteks ini. Nilai ROC-AUC juga meningkat dari 0.6866 menjadi 0.7880, menunjukkan bahwa model Decision Tree yang telah dioptimasi lebih baik dalam membedakan antara pelanggan yang churn dan tidak churn.

4.4 Pengujian Model Random Forest

Pengujian model Random Forest dilakukan dengan menggunakan data uji yang telah disiapkan. Model ini dilatih pada data latih dan kemudian diuji pada data uji untuk mengevaluasi performanya. Hasil dari pengujian ini akan memberikan gambaran tentang seberapa baik model dalam memprediksi churn pelanggan.

Setelah dilakukan pengujian model Random Forest, diperoleh hasil confusion matrix yang merepresentasikan performa model dalam mengklasifikasikan data uji. Confusion matrix ini menunjukkan jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Dari confusion matrix, dapat dihitung beberapa metrik evaluasi seperti akurasi, presisi, recall, dan F1-score.

Pengujian model Random Forest dipilih karena metode ini merupakan salah satu teknik ensemble yang menggunakan pendekatan bagging. Random Forest menggabungkan beberapa pohon keputusan untuk meningkatkan akurasi dan mengurangi risiko overfitting. Dengan menggunakan teknik ini, model dapat memanfaatkan kekuatan dari beberapa pohon keputusan yang berbeda, sehingga menghasilkan prediksi yang lebih stabil dan akurat.

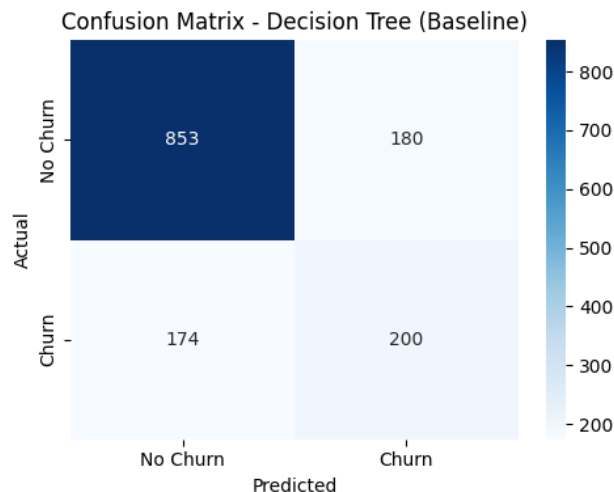
Perbandingan antara bagging dan boosting menjadi penting dalam konteks ini. Bagging, seperti yang diterapkan dalam Random Forest, berfokus pada pengurangan varians dengan membangun model independen secara paralel. Di sisi lain, boosting berusaha untuk mengurangi bias dengan membangun model secara berurutan, di mana setiap model baru berusaha untuk memperbaiki kesalahan dari model sebelumnya. Dengan membandingkan kedua pendekatan ini, kita dapat mengevaluasi kelebihan dan kekurangan masing-masing metode dalam konteks prediksi churn pelanggan.

Dengan demikian, pengujian Random Forest tidak hanya memberikan wawasan tentang performa model ini, tetapi juga memungkinkan kita untuk memahami perbedaan mendasar antara teknik bagging dan boosting dalam meningkatkan akurasi prediksi.

4.4.1 Hasil Random Forest tanpa hyperparameter tuning

Berikut adalah hasil confusion matrix dari model Random Forest yang telah diuji pada data uji, Gambar 21 menunjukkan confusion matrix yang diperoleh dari model Random Forest tanpa melakukan hyperparameter tuning. Sehingga model ini akan menjadi acuan untuk perbandingan dengan model-model lainnya yang akan diuji pada tahap selanjutnya. Namun kita tetap perlu menambahkan beberapa setting manual karena pada bab sebelumnya kita menemukan imbalance data pada dataset yang digunakan. Oleh karena itu, kita perlu mengatur beberapa parameter pada model Random Forest untuk mengatasi masalah tersebut. Parameter yang digunakan adalah sebagai berikut:

- **n_estimators=100**: Jumlah pohon (trees) yang dibangun dalam ensemble. Defaultnya adalah 100, semakin banyak pohon yang digunakan, prediksi menjadi lebih stabil tetapi waktu training juga lebih lama.
- **criterion="gini"**: Fungsi impurity yang digunakan untuk mengukur kualitas split pada setiap pohon. Alternatifnya adalah "entropy" atau "log_loss".
- **max_depth=None**: Kedalaman maksimum pohon tidak dibatasi (default), sehingga pohon akan tumbuh hingga semua daun bersifat *pure* atau tidak dapat di-split lagi. Hal ini berpotensi menimbulkan *overfitting*.
- **max_features="sqrt"**: Jumlah fitur yang dipertimbangkan pada setiap split. Defaultnya adalah "sqrt" untuk klasifikasi, yang berarti hanya menggunakan akar kuadrat dari jumlah total fitur.
- **class_weight="balanced"**: Memberikan bobot kelas secara otomatis yang proporsional terhadap kebalikannya dari frekuensi kelas, untuk mengatasi ketidakseimbangan kelas pada dataset.
- **random_state=42**: Menetapkan nilai *seed* untuk memastikan hasil dapat direproduksi.
- **n_jobs=-1**: Menggunakan semua core CPU yang tersedia untuk mempercepat proses training.



Gambar 21: Confusion Matrix Model Random Forest

Dari gambar 21 kita bisa identifikasi performa model Random Forest dalam mengklasifikasi data uji. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 853 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)

- False Positive (FP): 180 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 174 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 200 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Random Forest. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Random Forest:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{200 + 853}{200 + 853 + 180 + 174} = \frac{1053}{1407} \approx 0.749 \quad (27)$$

Akurasi model Random Forest adalah sekitar 74.9%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 74.9% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Random Forest:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{200}{200 + 180} = \frac{200}{380} \approx 0.526 \quad (28)$$

Presisi model Random Forest adalah sekitar 52.6%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 52.6% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Random Forest:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{200}{200 + 174} = \frac{200}{374} \approx 0.535 \quad (29)$$

Recall model Random Forest adalah sekitar 53.5%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 53.5% dari pelanggan yang sebenarnya churn, namun masih ada 46.5% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Random Forest:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.526 \cdot 0.535}{0.526 + 0.535} \approx 0.530 \quad (30)$$

F1-score model Random Forest adalah sekitar 53.0%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Random Forest adalah 0.7782249923642782. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.7782249923642782, model Random Forest ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.4.2 Hasil Random Forest dengan hyperparameter tuning

Setelah memperoleh hasil dari model Random Forest tanpa hyperparameter tuning, langkah selanjutnya adalah melakukan *hyperparameter tuning* untuk meningkatkan performa model, khususnya dalam hal *recall* yang menjadi fokus utama pada kasus prediksi *churn*. Proses *tuning* dilakukan menggunakan `RandomizedSearchCV` dengan skema *cross-validation*, di mana metrik utama yang digunakan sebagai acuan pemilihan model terbaik adalah *recall*.

Ruang pencarian (*search space*) untuk parameter model Random Forest dirancang dengan mempertimbangkan keseimbangan antara kompleksitas model dan risiko *overfitting*, yaitu sebagai berikut:

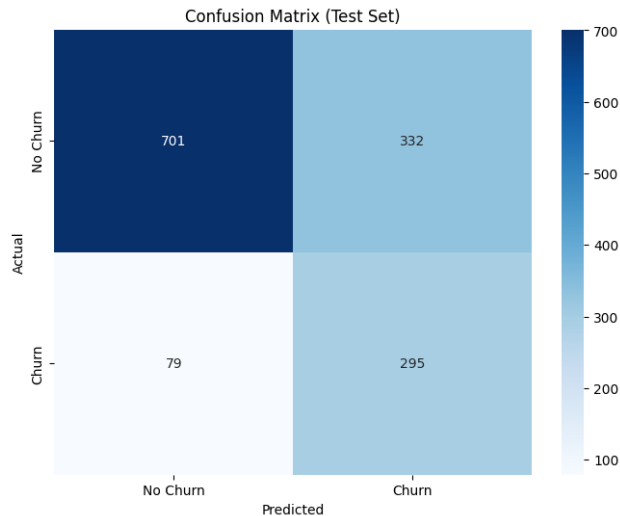
Ruang pencarian (*search space*) yang digunakan adalah sebagai berikut:

Table 8: Ruang Pencarian Hyperparameter Random Forest

Hyperparameter	Nilai yang diuji
n_estimators	200, 201, ..., 800
max_depth	5, 6, ..., 30
min_samples_split	2, 3, ..., 50
min_samples_leaf	1, 2, ..., 20
max_features	"sqrt", "log2"
class_weight	"balanced", "balanced_subsample"
bootstrap	True
criterion	"gini", "entropy", "log_loss"
ccp_alpha	[0.0, 0.01]

Dari tabel di atas, 80 iterasi `RandomizedSearchCV` dilakukan untuk mencari kombinasi hyperparameter yang optimal. Proses ini bertujuan untuk menemukan konfigurasi yang memberikan nilai *recall* tertinggi, sehingga model dapat lebih efektif dalam mendeteksi pelanggan yang berpotensi churn.

Setelah dilakukan `RandomizedSearchCV`, diperoleh confusion matrix dari model Random Forest yang telah dioptimasi hyperparameternya, seperti pada Gambar 22. Confusion matrix ini menunjukkan performa model dalam mengklasifikasikan data uji setelah proses tuning.



Gambar 22: Confusion Matrix Model Random Forest dengan Hyperparameter Tuning

Dari gambar 22, kita dapat mengidentifikasi performa model Random Forest yang telah dioptimasi hyperparameternya. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 701 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)

- False Positive (FP): 332 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 79 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 295 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model Random Forest yang telah dioptimasi hyperparameternya. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model Random Forest:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{295 + 701}{295 + 701 + 332 + 79} = \frac{996}{1407} \approx 0.708 \quad (31)$$

Akurasi model Random Forest adalah sekitar 70.8%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 70.8% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model Random Forest:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{295}{295 + 332} = \frac{295}{627} \approx 0.471 \quad (32)$$

Presisi model Random Forest adalah sekitar 47.1%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 47.1% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model Random Forest:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{295}{295 + 79} = \frac{295}{374} \approx 0.789 \quad (33)$$

Recall model Random Forest adalah sekitar 78.9%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 78.9% dari pelanggan yang sebenarnya churn, sehingga cukup efektif dalam mendeteksi churn meskipun masih ada 21.1% yang terlewat.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model Random Forest:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.471 \cdot 0.789}{0.471 + 0.789} \approx 0.589 \quad (34)$$

F1-score model Random Forest adalah sekitar 58.9%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, dengan keunggulan utama pada kemampuannya menangkap pelanggan churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model Random Forest yang telah dioptimasi hyperparameternya adalah 0.8216. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.8216, model Random Forest yang telah dioptimasi ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.4.3 Parameter Terbaik

Nilai-nilai di atas diperoleh dari proses `RandomizedSearchCV` yang telah dilakukan sebelumnya. Adapun kombinasi hyperparameter untuk mendapatkan hasil di atas ditunjukkan pada Tabel 9. Kombinasi ini merupakan hasil terbaik yang diperoleh dari proses tuning hyperparameter, yang bertujuan untuk meningkatkan performa model dalam mendeteksi pelanggan yang churn.

Table 9: Kombinasi Hyperparameter Terbaik (`RandomizedSearchCV`)

Hyperparameter	Nilai Terbaik
n_estimators	201
max_depth	9
min_samples_split	50
min_samples_leaf	17
max_features	sqrt
class_weight	balanced
ccp_alpha	0.006963042728397884
criterion	gini

4.4.4 Perbandingan Hasil Random Forest Sebelum dan Sesudah Hyperparameter Tuning

Setelah melakukan hyperparameter tuning, kita dapat membandingkan hasil model Random Forest sebelum dan sesudah tuning. Tabel 10 menunjukkan perbandingan metrik evaluasi antara model baseline dan model yang telah dioptimasi hyperparameter.

Table 10: Perbandingan Hasil Model Random Forest Sebelum dan Sesudah Hyperparameter Tuning

Metrik	Sebelum Tuning	Setelah Tuning
Akurasi	74.9%	70.8%
Presisi	52.6%	47.1%
Recall	53.5%	78.9%
F1-score	53.0%	58.9%
ROC-AUC	0.7782	0.8216

Dari tabel 10, dapat dilihat bahwa setelah melakukan hyperparameter tuning, model Random Forest mengalami peningkatan yang signifikan pada metrik recall, yaitu dari 53.5% menjadi 78.9%. Hal ini menunjukkan bahwa model yang telah dioptimasi lebih baik dalam mendeteksi pelanggan yang berpotensi churn.

Namun, akurasi dan presisi model mengalami penurunan, yaitu dari 74.9% menjadi 63.6% untuk akurasi dan dari 52.6% menjadi 41.3% untuk presisi. Meskipun demikian, peningkatan recall yang signifikan menunjukkan bahwa model ini lebih efektif dalam menangkap pelanggan churn, yang merupakan tujuan utama dari analisis ini.

Meskipun akurasi menurun, peningkatan recall menunjukkan bahwa model ini lebih fokus pada deteksi pelanggan churn, yang merupakan prioritas utama dalam konteks ini. Nilai ROC-AUC juga meningkat dari 0.7782 menjadi 0.8216, menunjukkan bahwa model Random Forest yang telah dioptimasi lebih baik dalam membedakan antara pelanggan yang churn dan tidak churn.

4.5 Pengujian Model XGBoost

Pengujian model XGBoost dilakukan dengan menggunakan data uji yang telah disiapkan. Model ini dilatih pada data latih dan kemudian diuji pada data uji untuk mengevaluasi performanya. Hasil dari pengujian ini akan memberikan gambaran tentang seberapa baik model dalam memprediksi churn pelanggan.

Setelah dilakukan pengujian model XGBoost, diperoleh hasil confusion matrix yang merepresentasikan performa model dalam mengklasifikasikan data uji. Confusion matrix ini menunjukkan jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Dari confusion matrix, dapat dihitung beberapa metrik evaluasi seperti akurasi, presisi, recall, dan F1-score.

Pengujian model XGBoost dipilih karena metode ini merupakan salah satu teknik boosting yang menggunakan pendekatan *gradient boosting*. XGBoost menggabungkan beberapa pohon keputusan secara sekuensial, di mana setiap pohon baru berusaha untuk memperbaiki

kesalahan dari pohon sebelumnya. Dengan menggunakan teknik ini, model dapat memanfaatkan kekuatan dari beberapa pohon keputusan yang berbeda, sehingga menghasilkan prediksi yang lebih stabil dan akurat.

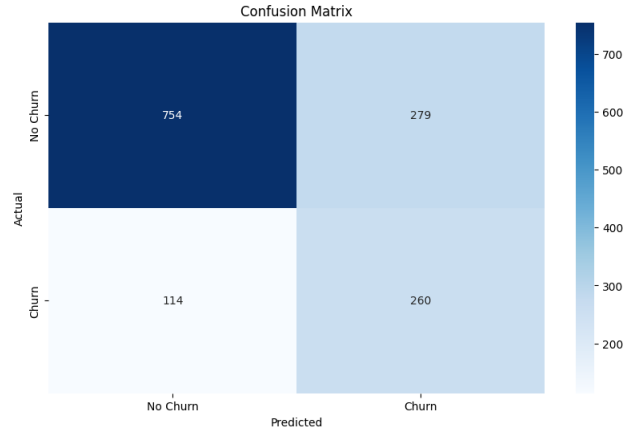
Perbandingan antara bagging dan boosting menjadi penting dalam konteks ini. Bagging, seperti yang diterapkan dalam Random Forest, berfokus pada pengurangan varians dengan membangun model independen secara paralel. Di sisi lain, boosting berusaha untuk mengurangi bias dengan membangun model secara berurutan, di mana setiap model baru berusaha untuk memperbaiki kesalahan dari model sebelumnya. Dengan membandingkan kedua pendekatan ini, kita dapat mengevaluasi kelebihan dan kekurangan masing-masing metode dalam konteks prediksi churn pelanggan.

Dengan demikian, pengujian XGBoost tidak hanya memberikan wawasan tentang performa model ini, tetapi juga memungkinkan kita untuk memahami perbedaan mendasar antara teknik bagging dan boosting dalam meningkatkan akurasi prediksi.

4.5.1 Hasil XGBoost tanpa hyperparameter tuning

Berikut adalah hasil confusion matrix dari model XGBoost yang telah diuji pada data uji, Gambar 23 menunjukkan confusion matrix yang diperoleh dari model XGBoost tanpa melakukan hyperparameter tuning. Sehingga model ini akan menjadi acuan untuk perbandingan dengan model-model lainnya yang akan diuji pada tahap selanjutnya. Namun kita tetap perlu menambahkan beberapa setting manual karena pada bab sebelumnya kita menemukan imbalance data pada dataset yang digunakan. Oleh karena itu, kita perlu mengatur beberapa parameter pada model XGBoost untuk mengatasi masalah tersebut. Parameter yang digunakan adalah sebagai berikut:

- **booster='gbtree'**: Booster yang digunakan, 'gbtree' paling umum untuk data tabular.
- **scale_pos_weight=3**: Bobot untuk mengatasi ketidakseimbangan kelas, dihitung sebagai rasio jumlah negatif terhadap jumlah positif.
- **random_state=42**: Menetapkan nilai seed untuk memastikan hasil dapat direproduksi.
- **n_jobs=-1**: Menggunakan semua core CPU yang tersedia untuk mempercepat proses training.



Gambar 23: Confusion Matrix Model XGBoost

Dari gambar 23 kita bisa identifikasi performa model XGBoost dalam mengklasifikasikan data uji. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 754 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)
- False Positive (FP): 279 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 114 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 260 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model XGBoost. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model XGBoost:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{260 + 754}{260 + 754 + 279 + 114} = \frac{1014}{1407} \approx 0.720 \quad (35)$$

Akurasi model XGBoost adalah sekitar 72.0%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 72.0% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model XGBoost:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{260}{260 + 279} = \frac{260}{539} \approx 0.482 \quad (36)$$

Presisi model XGBoost adalah sekitar 48.2%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 48.2% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model XGBoost:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{260}{260 + 114} = \frac{260}{374} \approx 0.695 \quad (37)$$

Recall model XGBoost adalah sekitar 69.5%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 69.5% dari pelanggan yang sebenarnya churn, namun masih ada 30.5% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model XGBoost:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.482 \cdot 0.695}{0.482 + 0.695} \approx 0.568 \quad (38)$$

F1-score model XGBoost adalah sekitar 56.8%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model XGBoost adalah 0.7908601705224905. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dengan nilai 0.7908601705224905, model XGBoost ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.5.2 Hasil XGBoost dengan hyperparameter tuning

Setelah memperoleh hasil dari model XGBoost tanpa hyperparameter tuning, langkah selanjutnya adalah melakukan *hyperparameter tuning* untuk meningkatkan performa model, khususnya dalam hal *recall* yang menjadi fokus utama pada kasus prediksi *churn*. Proses *tuning* dilakukan menggunakan `RandomizedSearchCV` dengan skema *cross-validation*, di mana metrik utama yang digunakan sebagai acuan pemilihan model terbaik adalah *recall*.

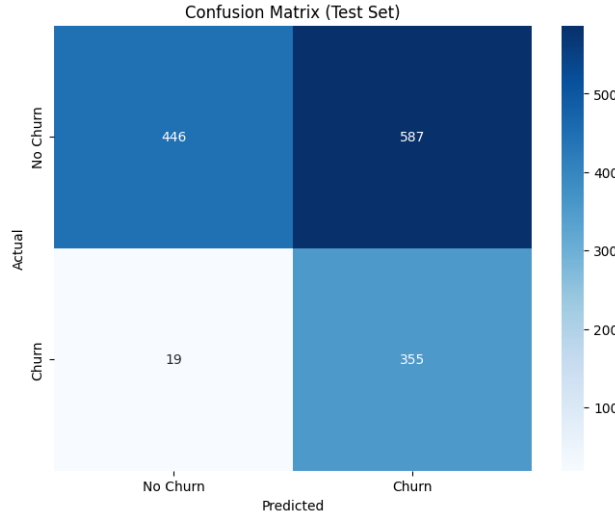
Ruang pencarian (search space) untuk parameter model XGBoost dirancang dengan mempertimbangkan keseimbangan antara kompleksitas model dan risiko overfitting, yaitu sebagai berikut:

Table 11: Ruang Pencarian Hyperparameter XGBoost

Hyperparameter	Nilai yang diuji
n_estimators	200, 201, ..., 1000
learning_rate	0.001, 0.002, ..., 0.3
max_depth	3, 4, ..., 10
min_child_weight	1, 2, ..., 10
subsample	0.5, 0.6, ..., 1.0
colsample_bytree	0.5, 0.6, ..., 1.0
gamma	10^{-8} , 10^{-7} , ..., 10^0
reg_alpha	10^{-8} , 10^{-7} , ..., 10^0
reg_lambda	10^{-3} , 10^{-2} , ..., 10^2
scale_pos_weight	[1.5, 2.0] (dengan spw base)

Dari tabel di atas, 80 iterasi `RandomizedSearchCV` dilakukan untuk mencari kombinasi hyperparameter yang optimal. Proses ini bertujuan untuk menemukan konfigurasi yang memberikan nilai *recall* tertinggi, sehingga model dapat lebih efektif dalam mendeteksi pelanggan yang berpotensi churn.

Setelah dilakukan `RandomizedSearchCV`, diperoleh confusion matrix dari model XGBoost yang telah dioptimasi hyperparameternya, seperti pada Gambar 24. Confusion matrix ini menunjukkan performa model dalam mengklasifikasikan data uji setelah proses tuning.



Gambar 24: Confusion Matrix Model XGBoost dengan Hyperparameter Tuning

Dari gambar 24, kita dapat mengidentifikasi performa model XGBoost yang telah dioptimasi hyperparameternya. Confusion matrix ini memberikan informasi tentang jumlah prediksi yang benar dan salah untuk masing-masing kelas (churn dan tidak churn). Detailnya adalah sebagai berikut:

- True Negative (TN): 446 (Pelanggan yang benar-benar tidak churn dan diprediksi tidak churn)
- False Positive (FP): 587 (Pelanggan yang tidak churn, tetapi diprediksi churn)
- False Negative (FN): 19 (Pelanggan yang churn, tetapi diprediksi tidak churn)
- True Positive (TP): 355 (Pelanggan yang benar-benar churn dan diprediksi churn)

Setelah mendapatkan confusion matrix, kita dapat menghitung beberapa metrik evaluasi untuk model XGBoost yang telah dioptimasi hyperparameternya. Metrik yang umum digunakan adalah akurasi, presisi, recall, dan F1-score.

Akurasi

Akurasi adalah proporsi prediksi yang benar dari total prediksi yang dilakukan oleh model. Berikut adalah perhitungan akurasi model XGBoost setelah hyperparameter tuning:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{355 + 446}{355 + 446 + 587 + 19} = \frac{801}{1407} \approx 0.569 \quad (39)$$

Akurasi model XGBoost setelah hyperparameter tuning adalah sekitar 56.9%. Ini menunjukkan bahwa model ini mampu memprediksi dengan benar sekitar 56.9% dari total data uji yang diberikan.

Precision

Precision adalah proporsi prediksi positif yang benar dari total prediksi positif yang dilakukan oleh model. Berikut adalah perhitungan presisi model XGBoost setelah hyperparameter tuning:

$$\text{Presisi} = \frac{TP}{TP + FP} = \frac{355}{355 + 587} = \frac{355}{942} \approx 0.377 \quad (40)$$

Presisi model XGBoost setelah hyperparameter tuning adalah sekitar 37.7%. Ini menunjukkan bahwa dari semua pelanggan yang diprediksi churn, hanya sekitar 37.7% yang benar-benar churn. Hal ini mengindikasikan bahwa model ini masih menghasilkan cukup banyak false positive, yaitu pelanggan yang diprediksi churn padahal sebenarnya tidak.

Recall

Recall adalah proporsi prediksi positif yang benar dari total aktual positif. Berikut adalah perhitungan recall model XGBoost setelah hyperparameter tuning:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{355}{355 + 19} = \frac{355}{374} \approx 0.949 \quad (41)$$

Recall model XGBoost setelah hyperparameter tuning adalah sekitar 94.9%. Ini menunjukkan bahwa model ini mampu menangkap sekitar 94.9% dari pelanggan yang sebenarnya churn, namun masih ada 5.1% pelanggan churn yang tidak terdeteksi oleh model.

F1-score

F1-score adalah rata-rata harmonis dari presisi dan recall. F1-score memberikan gambaran yang lebih baik tentang keseimbangan antara presisi dan recall. Berikut adalah perhitungan F1-score model XGBoost setelah hyperparameter tuning:

$$\text{F1-score} = 2 \cdot \frac{\text{Presisi} \cdot \text{Recall}}{\text{Presisi} + \text{Recall}} = 2 \cdot \frac{0.377 \cdot 0.949}{0.377 + 0.949} \approx 0.528 \quad (42)$$

F1-score model XGBoost setelah hyperparameter tuning adalah sekitar 52.8%. Ini menunjukkan bahwa model ini memiliki keseimbangan yang cukup baik antara presisi dan recall, meskipun masih ada ruang untuk perbaikan, terutama dalam hal menangkap pelanggan yang churn.

ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Adapun nilai ROC-AUC yang didapatkan dari model XGBoost setelah hyperparameter tuning adalah 0.8276. Nilai ini menunjukkan bahwa model memiliki kemampuan yang cukup baik dalam membedakan antara pelanggan yang churn dan tidak churn.

Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna.

Dengan nilai 0.8276, model XGBoost ini menunjukkan performa yang cukup baik dalam memprediksi churn pelanggan.

4.5.3 Parameter Terbaik

Nilai-nilai di atas diperoleh dari proses `RandomizedSearchCV` yang telah dilakukan sebelumnya. Adapun kombinasi hyperparameter untuk mendapatkan hasil di atas ditunjukkan pada Tabel 12. Kombinasi ini merupakan hasil terbaik yang diperoleh dari proses tuning hyperparameter, yang bertujuan untuk meningkatkan performa model dalam mendeteksi pelanggan yang churn.

Table 12: Kombinasi Hyperparameter Terbaik (`RandomizedSearchCV`)

Hyperparameter	Nilai Terbaik
<code>colsample_bytree</code>	0.8006
<code>gamma</code>	0.0046
<code>learning_rate</code>	0.0011
<code>max_depth</code>	4
<code>min_child_weight</code>	8
<code>n_estimators</code>	691
<code>reg_alpha</code>	0.3224
<code>reg_lambda</code>	0.0010
<code>scale_pos_weight</code>	5.5251
<code>subsample</code>	0.5917

4.5.4 Perbandingan Hasil XGBoost Sebelum dan Sesudah Hyperparameter Tuning

Setelah melakukan hyperparameter tuning, kita dapat membandingkan hasil model XGBoost sebelum dan sesudah tuning. Tabel 13 menunjukkan perbandingan metrik evaluasi antara model baseline dan model yang telah dioptimasi hyperparameter.

Table 13: Perbandingan Hasil Model XGBoost Sebelum dan Sesudah Hyperparameter Tuning

Metrik	Sebelum Tuning	Setelah Tuning
Akurasi	72.0%	56.9%
Presisi	48.2%	37.7%
Recall	69.5%	94.9%
F1-score	56.8%	52.8%
ROC-AUC	0.7909	0.8276

Dari tabel 13, dapat dilihat bahwa setelah melakukan hyperparameter tuning, model XGBoost mengalami peningkatan yang signifikan pada metrik recall, yaitu dari 69.5% menjadi 94.9%. Hal ini menunjukkan bahwa model yang telah dioptimasi lebih baik dalam mendeteksi pelanggan yang berpotensi churn.

Namun, akurasi dan presisi model mengalami penurunan, yaitu dari 72.0% menjadi 56.9% untuk akurasi dan dari 48.2% menjadi 37.7% untuk presisi. Meskipun demikian, peningkatan recall yang signifikan menunjukkan bahwa model ini lebih efektif dalam menangkap pelanggan churn, yang merupakan tujuan utama dari analisis ini.

Meskipun akurasi menurun, peningkatan recall menunjukkan bahwa model ini lebih fokus pada deteksi pelanggan churn, yang merupakan prioritas utama dalam konteks ini. Nilai ROC-AUC juga meningkat dari 0.7909 menjadi 0.8276, menunjukkan bahwa model XGBoost yang telah dioptimasi lebih baik dalam membedakan antara pelanggan yang churn dan tidak churn.

4.6 Perbandingan Model Logistic Regression, Decision Tree, Random Forest, dan XGBoost Sebelum dan Sesudah Hyperparameter Tuning

Setelah melakukan pengujian dan evaluasi pada model Logistic Regression, Decision Tree, Random Forest, dan XGBoost, kita dapat membandingkan hasil dari masing-masing model sebelum dan sesudah hyperparameter tuning. Tabel 14 menunjukkan perbandingan metrik evaluasi antara keempat model tersebut.

Table 14: Perbandingan Hasil Model Sebelum dan Sesudah Hyperparameter Tuning

Metrik	Logistic Regression	Decision Tree	Random Forest	XGBoost
Akurasi	72.4% / 71.7%	72.4% / 63.6%	74.9% / 70.8%	72.0% / 56.9%
Presisi	48.8% / 48.1%	48.5% / 41.3%	52.6% / 47.1%	48.2% / 37.7%
Recall	79.1% / 80.4%	58.8% / 87.2%	53.5% / 78.9%	69.5% / 94.9%
F1-score	60.4% / 60.4%	53.1% / 56.1%	53.0% / 58.9%	56.8% / 52.8%
ROC-AUC	0.8279 / 0.8279	0.6866 / 0.7880	0.7782 / 0.8216	0.7909 / 0.8276

Nilai sebelum tanda garis miring (/) adalah hasil sebelum hyperparameter tuning, sedangkan nilai setelah tanda garis miring adalah hasil setelah hyperparameter tuning.

Dari tabel 14, kita dapat melihat perbandingan performa masing-masing model sebelum dan sesudah hyperparameter tuning. Pertama tama kita akan analisa kenaikan dan 2 metric utama yang diperhatikan yaitu recall dan ROC-AUC.

4.6.1 Analisis Nilai Recall

Dari tabel 14, kita dapat membandingkan recall yang dihasilkan dari setiap model sebelum dan sesudah hyperparameter tuning. Recall adalah metrik yang sangat penting dalam konteks prediksi churn, karena menunjukkan seberapa baik model dalam menangkap pelanggan yang sebenarnya churn. Berikut adalah rincian recall untuk masing-masing model:

- **Logistic Regression:** Recall tertinggi yang didapatkan adalah model Logistic Regression setelah hyperparameter tuning, yaitu 80.4%.
- **Decision Tree:** Recall tertinggi yang didapatkan adalah model Decision Tree setelah hyperparameter tuning, yaitu 87.2%.

- **Random Forest:** Recall tertinggi yang didapatkan adalah model Random Forest setelah hyperparameter tuning, yaitu 78.9%.
- **XGBoost:** Recall tertinggi yang didapatkan adalah model XGBoost setelah hyperparameter tuning, yaitu 94.9%.

Hal tersebut menunjukkan bahwa model XGBoost memiliki recall tertinggi, yaitu 94.9%, yang menunjukkan bahwa model ini sangat efektif dalam mendeteksi pelanggan yang berpotensi churn. Hasil eksperimen menunjukkan bahwa metode boosting (XGBoost) secara konsisten memberikan kinerja recall yang lebih tinggi dibandingkan metode bagging (Random Forest) maupun model tunggal (Decision Tree, Logistic Regression). Hal ini konsisten dengan karakteristik boosting yang fokus memperbaiki kesalahan prediksi sebelumnya, sehingga lebih efektif dalam mendeteksi pelanggan churn.

4.6.2 Analisis Kenaikan Recall

Dari tabel 14, kita dapat melihat bahwa semua model mengalami peningkatan yang signifikan pada metrik recall setelah melakukan hyperparameter tuning. Berikut adalah rincian kenaikan recall untuk masing-masing model:

- **Logistic Regression:** Recall meningkat dari 79.1% menjadi 80.4%, dengan kenaikan sebesar 1.3%.
- **Decision Tree:** Recall meningkat dari 58.8% menjadi 87.2%, dengan kenaikan yang sangat signifikan sebesar 28.4%.
- **Random Forest:** Recall meningkat dari 53.5% menjadi 78.9%, dengan kenaikan sebesar 25.4%.
- **XGBoost:** Recall meningkat dari 69.5% menjadi 94.9%, dengan kenaikan yang sangat signifikan sebesar 25.4%.

Dari analisis di atas, kita dapat melihat bahwa model Decision Tree, Random Forest, dan XGBoost mengalami peningkatan recall yang paling signifikan, yaitu masing-masing sebesar 28.4%, dan 25.4% baik pada Random Forest dan XGBoost. Hal ini bahwa pengaruh hyperparameter tuning sangat besar dalam meningkatkan kemampuan model untuk mendeteksi pelanggan yang churn. Meskipun Logistic Regression juga mengalami peningkatan, namun kenaikannya tidak sebesar model lainnya.

4.7 Analisis Nilai ROC-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) adalah metrik yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Nilai ROC-AUC berkisar antara 0 hingga 1, di mana nilai 0.5 menunjukkan model yang tidak lebih baik dari tebakan acak, sedangkan nilai 1 menunjukkan model yang sempurna. Dalam konteks ini, kita akan menganalisis nilai ROC-AUC dari masing-masing model sebelum dan sesudah hyperparameter tuning.

- **Logistic Regression:** ROC-AUC terbaik tetap pada kedua jenis model logistic regression yaitu sebesar 0.8279, tidak ada perubahan.
- **Decision Tree:** ROC-AUC tertinggi yang didapatkan adalah model Decision Tree setelah hyperparameter tuning, yaitu 0.7880.
- **Random Forest:** ROC-AUC tertinggi yang didapatkan adalah model Random Forest setelah hyperparameter tuning, yaitu 0.8216.
- **XGBoost:** ROC-AUC tertinggi yang didapatkan adalah model XGBoost setelah hyperparameter tuning, yaitu 0.8276.

Dari analisis di atas, kita dapat melihat bahwa model XGBoost memiliki nilai ROC-AUC tertinggi, yaitu 0.8276, yang menunjukkan bahwa model ini sangat baik dalam membedakan antara pelanggan yang churn dan tidak churn. Hal ini konsisten dengan hasil recall yang diperoleh sebelumnya, di mana model XGBoost juga menunjukkan performa terbaik dalam mendeteksi pelanggan churn.

4.7.1 Analisis Kenaikan ROC-AUC

Dari tabel 14, kita juga dapat melihat peningkatan nilai ROC-AUC untuk semua model setelah melakukan hyperparameter tuning. Berikut adalah rincian kenaikan ROC-AUC untuk masing-masing model:

- **Logistic Regression:** ROC-AUC tetap pada 0.8279, tidak ada perubahan.
- **Decision Tree:** ROC-AUC meningkat dari 0.6866 menjadi 0.7880, dengan kenaikan sebesar 0.1014.
- **Random Forest:** ROC-AUC meningkat dari 0.7782 menjadi 0.8216, dengan kenaikan sebesar 0.0434.
- **XGBoost:** ROC-AUC meningkat dari 0.7909 menjadi 0.8276, dengan kenaikan sebesar 0.0367.

Dari analisis di atas, kita dapat melihat bahwa model Decision Tree mengalami peningkatan ROC-AUC yang paling signifikan, yaitu sebesar 0.1014. Hal ini menunjukkan bahwa model ini lebih baik dalam membedakan antara pelanggan yang churn dan tidak churn setelah dilakukan hyperparameter tuning. Sementara itu, model Random Forest dan XGBoost juga mengalami peningkatan ROC-AUC, meskipun tidak sebesar Decision Tree.

4.8 Kesimpulan Perbandingan Model

Dari perbandingan hasil model Logistic Regression, Decision Tree, Random Forest, dan XGBoost sebelum dan sesudah hyperparameter tuning, kita dapat menarik beberapa kesimpulan:

- **Model XGBoost** menunjukkan performa terbaik dalam hal recall, yaitu 94.9%, yang menunjukkan bahwa model ini sangat efektif dalam mendeteksi pelanggan yang berpotensi churn. Selain itu, nilai ROC-AUC juga tinggi, yaitu 0.8276, menunjukkan kemampuan model dalam membedakan antara pelanggan yang churn dan tidak churn.
- **Model Decision Tree** mengalami peningkatan recall yang signifikan setelah hyperparameter tuning, yaitu dari 58.8% menjadi 87.2%. Meskipun nilai ROC-AUC-nya tidak setinggi XGBoost, peningkatan ini menunjukkan bahwa model ini dapat dioptimasi lebih lanjut untuk meningkatkan performanya.
- **Model Random Forest** juga menunjukkan peningkatan recall yang signifikan, yaitu dari 53.5% menjadi 78.9%. Nilai ROC-AUC-nya juga meningkat, meskipun tidak sebesar Decision Tree.
- **Model Logistic Regression** memiliki performa yang stabil dengan recall sekitar 80.4% setelah hyperparameter tuning, namun tidak mengalami peningkatan yang signifikan dibandingkan model lainnya.
- Secara keseluruhan, model XGBoost adalah yang paling unggul dalam hal deteksi churn, diikuti oleh Decision Tree dan Random Forest. Logistic Regression memiliki performa yang baik, namun tidak sebaik model-model lainnya dalam konteks ini.
- Hyperparameter tuning terbukti efektif dalam meningkatkan performa model, terutama pada model Decision Tree, Random Forest, dan XGBoost. Hal ini menunjukkan pentingnya proses tuning untuk mendapatkan model yang optimal dalam konteks prediksi churn.
- Meskipun Logistic Regression memiliki performa yang baik, model ini mungkin tidak cukup kompleks untuk menangkap pola yang lebih rumit dalam data churn. Oleh karena itu, model berbasis pohon keputusan seperti Decision Tree, Random Forest, dan XGBoost lebih unggul dalam konteks ini.
- Metode boosting (XGBoost) secara konsisten memberikan kinerja recall yang lebih tinggi dibandingkan metode bagging (Random Forest) maupun model tunggal (Decision Tree, Logistic Regression). Hal ini konsisten dengan karakteristik boosting yang fokus memperbaiki kesalahan prediksi sebelumnya, sehingga lebih efektif dalam mendeteksi pelanggan churn.

Dengan demikian, model XGBoost adalah pilihan terbaik untuk kasus prediksi churn ini, terutama setelah dilakukan hyperparameter tuning yang signifikan meningkatkan performa model. Model ini dapat digunakan untuk membantu perusahaan dalam mengidentifikasi pelanggan yang berpotensi churn dan mengambil tindakan yang tepat untuk mempertahankan mereka.

4.9 Implikasi Bisnis

Dalam konteks bisnis, hasil analisis ini memiliki implikasi yang signifikan bagi perusahaan dalam mengelola churn pelanggan. Berikut adalah beberapa poin penting yang dapat diambil dari hasil analisis ini:

- **Identifikasi Pelanggan Berisiko Churn:** Model XGBoost yang telah dioptimasi dapat digunakan untuk mengidentifikasi pelanggan yang berisiko tinggi untuk churn. Dengan mengetahui pelanggan mana yang berpotensi churn, perusahaan dapat mengambil tindakan proaktif untuk mempertahankan mereka.
- **Strategi Pemasaran yang Lebih Efektif:** Dengan informasi tentang pelanggan yang berisiko churn, perusahaan dapat merancang strategi pemasaran yang lebih efektif, seperti menawarkan diskon atau promosi khusus kepada pelanggan tersebut untuk mendorong mereka tetap menggunakan layanan.
- **Peningkatan Layanan Pelanggan:** Hasil analisis ini juga dapat digunakan untuk meningkatkan layanan pelanggan. Perusahaan dapat fokus pada peningkatan pengalaman pelanggan bagi mereka yang berisiko churn, sehingga dapat mengurangi tingkat churn.
- **Pengalokasian Sumber Daya yang Lebih Baik:** Dengan mengetahui pelanggan mana yang berisiko churn, perusahaan dapat mengalokasikan sumber daya dengan lebih efisien. Misalnya, perusahaan dapat mengarahkan tim layanan pelanggan untuk fokus pada pelanggan yang berisiko tinggi.
- **Peningkatan Retensi Pelanggan:** Dengan menerapkan strategi berdasarkan hasil analisis ini, perusahaan diharapkan dapat meningkatkan retensi pelanggan dan mengurangi tingkat churn. Hal ini akan berdampak positif pada pendapatan dan profitabilitas perusahaan.
- **Pengembangan Produk dan Layanan Baru:** Hasil analisis ini juga dapat memberikan wawasan tentang kebutuhan dan preferensi pelanggan. Dengan memahami faktor-faktor yang mempengaruhi churn, perusahaan dapat mengembangkan produk atau layanan baru yang lebih sesuai dengan kebutuhan pelanggan.

5 Kesimpulan dan Saran

Adapun 3 jenis kesimpulan yang bisa diambil dari tugas ini, diantaranya kesimpulan pada proses EDA, kesimpulan pada evaluasi model, dan kesimpulan bisnis.

5.1 Kesimpulan EDA

Berikut adalah kesimpulan dari tahap EDA :

1. Pelanggan dengan kontrak "Month-to-month" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang bekerja dengan kontrak tahunan atau dua tahunan.
2. Layanan TechSupport dapat membantu mengurangi tingkat churn pelanggan.
3. Layanan OnlineSecurity dapat membantu mengurangi tingkat churn pelanggan.
4. Pelanggan yang menggunakan layanan internet "Fiber optic" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan layanan DSL atau tidak menggunakan layanan internet.
5. Pelanggan yang menggunakan metode pembayaran "Electronic check" cenderung lebih sering berhenti berlangganan dibandingkan dengan pelanggan yang menggunakan metode pembayaran lainnya.
6. Pelanggan dengan lama berlangganan yang lebih lama cenderung memiliki total biaya yang lebih tinggi.
7. Pelanggan dengan biaya bulanan yang lebih tinggi cenderung memiliki total biaya yang lebih tinggi.
8. Terdapat segmen highend pada fitur TotalCharges menandakan adanya pelanggan yang memiliki total biaya yang sangat tinggi.
9. Pelanggan dengan biaya bulanan yang lebih tinggi cenderung untuk churn.

5.2 Kesimpulan Evaluasi Model

Berikut adalah kesimpulan dari tahap evaluasi model :

1. **Model XGBoost** menunjukkan performa terbaik dalam hal recall, yaitu 94.9%, yang menunjukkan bahwa model ini sangat efektif dalam mendeteksi pelanggan yang berpotensi churn. Selain itu, nilai ROC-AUC juga tinggi, yaitu 0.8276, menunjukkan kemampuan model dalam membedakan antara pelanggan yang churn dan tidak churn.
2. **Model Decision Tree** mengalami peningkatan recall yang signifikan setelah hyperparameter tuning, yaitu dari 58.8% menjadi 87.2%. Meskipun nilai ROC-AUC-nya tidak setinggi XGBoost, peningkatan ini menunjukkan bahwa model ini dapat dioptimasi lebih lanjut untuk meningkatkan performanya.

3. **Model Random Forest** juga menunjukkan peningkatan recall yang signifikan, yaitu dari 53.5% menjadi 78.9%. Nilai ROC-AUC-nya juga meningkat, meskipun tidak sebesar Decision Tree.
4. **Model Logistic Regression** memiliki performa yang stabil dengan recall sekitar 80.4% setelah hyperparameter tuning, namun tidak mengalami peningkatan yang signifikan dibandingkan model lainnya.
5. Secara keseluruhan, model XGBoost adalah yang paling unggul dalam hal deteksi churn, diikuti oleh Decision Tree dan Random Forest. Logistic Regression memiliki performa yang baik, namun tidak sebaik model-model lainnya dalam konteks ini.
6. Hyperparameter tuning terbukti efektif dalam meningkatkan performa model, terutama pada model Decision Tree, Random Forest, dan XGBoost. Hal ini menunjukkan pentingnya proses tuning untuk mendapatkan model yang optimal dalam konteks prediksi churn.
7. Meskipun Logistic Regression memiliki performa yang baik, model ini mungkin tidak cukup kompleks untuk menangkap pola yang lebih rumit dalam data churn. Oleh karena itu, model berbasis pohon keputusan seperti Decision Tree, Random Forest, dan XGBoost lebih unggul dalam konteks ini.
8. Metode boosting (XGBoost) secara konsisten memberikan kinerja recall yang lebih tinggi dibandingkan metode bagging (Random Forest) maupun model tunggal (Decision Tree, Logistic Regression). Hal ini konsisten dengan karakteristik boosting yang fokus memperbaiki kesalahan prediksi sebelumnya, sehingga lebih efektif dalam mendeteksi pelanggan churn.

5.3 Kesimpulan Bisnis

Berikut adalah kesimpulan bisnis yang dapat diambil dari hasil analisis ini :

1. **Identifikasi Pelanggan Berisiko Churn:** Model XGBoost yang telah dioptimasi dapat digunakan untuk mengidentifikasi pelanggan yang berisiko tinggi untuk churn. Dengan mengetahui pelanggan mana yang berpotensi churn, perusahaan dapat mengambil tindakan proaktif untuk mempertahankan mereka.
2. **Strategi Pemasaran yang Lebih Efektif:** Dengan informasi tentang pelanggan yang berisiko churn, perusahaan dapat merancang strategi pemasaran yang lebih efektif, seperti menawarkan diskon atau promosi khusus kepada pelanggan tersebut untuk mendorong mereka tetap menggunakan layanan.
3. **Peningkatan Layanan Pelanggan:** Hasil analisis ini juga dapat digunakan untuk meningkatkan layanan pelanggan. Perusahaan dapat fokus pada peningkatan pengalaman pelanggan bagi mereka yang berisiko churn, sehingga dapat mengurangi tingkat churn.

4. **Pengalokasian Sumber Daya yang Lebih Baik:** Dengan mengetahui pelanggan mana yang berisiko churn, perusahaan dapat mengalokasikan sumber daya dengan lebih efisien. Misalnya, perusahaan dapat mengarahkan tim layanan pelanggan untuk fokus pada pelanggan yang berisiko tinggi.
5. **Peningkatan Retensi Pelanggan:** Dengan menerapkan strategi berdasarkan hasil analisis ini, perusahaan diharapkan dapat meningkatkan retensi pelanggan dan mengurangi tingkat churn. Hal ini akan berdampak positif pada pendapatan dan profitabilitas perusahaan.
6. **Pengembangan Produk dan Layanan Baru:** Hasil analisis ini juga dapat memberikan wawasan tentang kebutuhan dan preferensi pelanggan. Dengan memahami faktor-faktor yang mempengaruhi churn, perusahaan dapat mengembangkan produk atau layanan baru yang lebih sesuai dengan kebutuhan pelanggan.